arXiv:2605.23872v1 [cs.LG] 22 May 2026

免训练循环变换器

陈立章 * **李乔纳森** * **陈亮 倪老 刘强**

摘要

本文提出免训练循环变换器，其中轻量级推理时包装器循环冻结检查点的连续中间层块，无需额外微调、继续训练或架构更改。与先前端到端训练循环结构的循环变换器方法不同，我们在测试时将循环性改装到预训练模型上。我们表明，朴素的块重复应用通常会降低性能，凸显了循环应用策略的重要性。受将预归一化变换器块视为常微分方程上前向欧拉步骤的启发，我们转而将循环视为对同一近似的细化，用较小的阻尼子步骤取代一次大的更新。在七个稠密、稀疏专家混合以及多头潜在注意力加专家混合模型家族中，我们的方法在 MMLU-Pro 上将 Qwen3-4B-Instruct 提升 +2.64 个百分点，在 CommonsenseQA 上将 Qwen3-30B-A3B-Instruct 提升 +1.14 个百分点，在 OpenBookQA 上将 Moonlight-16B-A3B-Instruct 提升 +1.20 个百分点。

1 引言

循环变换器 [35, 28]、通用变换器 [27, 47] 和深度均衡模型 [6, 7] 都将循环性融入架构 [95, 77, 61, 36] 和权重 [33, 102, 74, 94] 中。层在循环迭代间绑定并相应优化，因此循环结构与训练参数密不可分。因此，循环无法应用于未按此方法论训练的模型，而几乎所有公开发布的语言模型检查点都是如此。这自然引出了本文的核心问题：

> 能否在推理时直接循环一个冻结的现成检查点，无需微调、无需继续训练、无需辅助参数、也无需架构更改？

具体而言，现代开源权重大语言模型，如 Qwen3[86]、Llama-3.2[83]、Moonlight[55] 和 DeepSeek-V2-Lite[25]，是多阶段流程的终点，该流程通常包括继续预训练、监督微调以及一轮或多轮基于人类反馈的强化学习或直接偏好优化后训练 [26]。我们采用模型作者发布的任何权重，并在推理时应用循环包装器，不进行任何形式的进一步权重更新。

多条汇聚的证据线索指向核心问题的肯定答案。[59] 表明，已发布变换器语言模型的整个中间层块可以在损失极小的情况下删除；[46] 报告称，中间层可以被跳过、交换或重复而不会造成灾难性退化；[10] 证明，中间层逻辑值已经编码了最终预测的大部分信息 [80, 75]。对这些结果的主流解释是可压缩性——中间层是冗余的，因此可以移除——但这忽略了一个互补的视角：使中间层可以安全删除的冗余性，同样使其可以安全地重复应用。这一洞见引出了我们在第 2.3 节中发展的数值分析解释：每个预归一化变换器层恰好是一个前向

* 等贡献 德克萨斯大学奥斯汀分校

预印本。

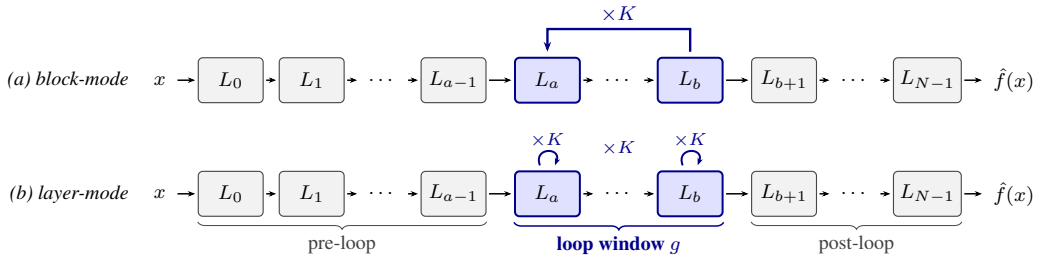


图 1：无需训练的循环变换器包装器，两种迭代模式。在推理时，通过在恢复循环后层之前重复应用一个连续的中间块 $g = L_b \circ \cdots \circ L_a$ 进行 K 次迭代，来增强冻结的检查点。不改变任何权重，也不引入新参数；代价是在循环窗口内进行 $(b-a+1)(K-1)$ 次额外的前向传递。(a) 块模式对整个窗口迭代 K 次，$(L_b \circ \cdots \circ L_a)^K$. (b) 层模式在传递之前对每个窗口层迭代 K 次，$L_b^K \circ \cdots \circ L_a^K$，这是在专家混合骨干上更安全的默认设置，因为它在迭代过程中固定了每层的专家路由（第 2.2 节）。第 2.3 节将 $g^{(K)}$ 解释为对网络已经用单步前向欧拉方法积分的残差常微分方程进行子步进。

在 $h{=}1$ 处对每个块的残差常微分方程进行欧拉步进，因此在推理时重复应用该块，从几何上讲，是对网络已经近似的常微分方程进行更精细的积分 [6, 7]。特别地，大小为 $h{=}1/K$ 的 K 个子步更好地近似了未修改网络被训练来提供的相同 $t{=}1$ 端点。

我们表明，这一几何动机转化为在七个模型家族——Qwen3 [86]（0.6B/1.7B/4B 基础版与指令版，30B-A3B 混合专家）、Qwen1.5-MoE-A2.7B、Llama-3.2 [83]（1B/3B）、Moonlight [55] 和 DeepSeek-V2-Lite [25]——以及 45 个（模型，基准）单元的无训练循环评估中可衡量的收益。循环评估在知识密集型多项选择基准上收益最大：Qwen3-4B-Instruct 在 MMLU-Pro [91] 上提升 +2.64，在 GPQA-Main [76] 上提升 +2.01，Qwen1.5-MoE-A2.7B-Chat 在 ARC-Challenge [21] 上提升 +2.30。实验使用了约 20,000 个 NVIDIA H100 HBM3-80GB GPU 小时。

这些改进无需任何参数更新、额外监督或针对特定基准的调优，表明重复应用现有变换器模块可以释放潜在的推理时计算。我们进一步将前向欧拉积分与高阶不动点加速器和求解器进行比较，包括 Anderson 加速、重球法、Aitken 加速 [89] 和 Runge – Kutta 风格更新。

我们的主要贡献如下：

1. 一个完全指定的无训练循环包装器，具有块和层迭代模式以及七种循环迭代策略，包括变换器隐式近似的常微分方程的著名数值积分方法。2. 在 7 个模型家族和 45 个（模型，基准）单元上，使用单一开箱即用配方（K -阶段 Runge – Kutta 在中间 4 层；密集模型使用块模式，混合专家模型使用层模式）进行跨架构验证，没有任何形式的逐单元超参数调优。3. 对于混合专家检查点 [23, 4, 5]，层模式迭代：$L_b^K \circ \cdots \circ L_a^K$ 而非 $(L_b \circ \cdots \circ L_a)^K$ 是必要的，因为块模式会导致专家路由在迭代之间剧烈波动。

2 无训练循环变换器

2.1 循环包装器

设 $f = L_{N-1} \circ \cdots \circ L_0$ 表示一个具有 N 个解码器层 [87, 86, 83, 25, 55] 的预训练变换器，其中 L_i 将形状为 $\mathbb{R}^{T \times d}$ 的残差流映射到自身。选择一个由 $[a, b]$ 索引的连续循环窗口，其中 $0 \le a \le b \le N-1$，以及循环次数 $K \ge 1$。该窗口诱导出一个算子

$$g := L_b \circ L_{b-1} \circ \cdots \circ L_a, \qquad g : \mathbb{R}^{T \times d} \to \mathbb{R}^{T \times d}. \tag{1}$$

Algorithm 1 **Block-mode** Runge–Kutta forward pass.

Input: x, block range $[a, b]$, number of stages s, Runge–Kutta coefficients $\{a_{ij}\}_{1\le j<i\le s}, \{b_i\}_{i=1}^{s}$

$x \leftarrow \textsc{PreLoop}(x)$ ▷ *embedding, pre-block, etc.*
$y_0 \leftarrow x$ ▷ *initial state for the block*
for $i = 1, \dots, s$ **do** ▷ *iterate over the s Runge–Kutta stages*
  $y_i \leftarrow y_0 + \sum_{j<i} a_{ij} k_j$ ▷ *add weighted increments*
  $k_i \leftarrow (L_b \circ \cdots \circ L_a)(y_i) - y_i$ ▷ *residual of the full composed block*
end for
$x \leftarrow y_0 + \sum_{i=1}^{s} b_i k_i$ ▷ *s^{th} order Runge–Kutta update*
return $\textsc{PostLoop}(x)$ ▷ *layers after the loop block, output head*

Algorithm 2 **Layer-mode** Runge–Kutta forward pass.

Input: x, block range $[a, b]$, number of stages s, Runge–Kutta coefficients $\{a_{ij}\}_{1\le j<i\le s}, \{b_i\}_{i=1}^{s}$

$x \leftarrow \textsc{PreLoop}(x)$ ▷ *embedding, pre-block, etc.*
for $\ell = a, \dots, b$ **do** ▷ *outer loop over each layer*
  $y_0 \leftarrow x$ ▷ *initial state for every layer*
  for $i = 1, \dots, s$ **do** ▷ *iterate over the s Runge–Kutta stages*
    $y_i \leftarrow y_0 + \sum_{j<i} a_{ij} k_j$ ▷ *add weighted increments*
    $k_i \leftarrow L_\ell(y_i) - y_i$ ▷ *residual of a single layer L_ℓ*
  end for
  $x \leftarrow y_0 + \sum_{i=1}^{s} b_i k_i$ ▷ *s^{th} order Runge–Kutta update*
end for
return $\textsc{PostLoop}(x)$ ▷ *layers after the loop block, output head*

循环包装器将网络分为循环前层 $0, \dots, a-1$ 、循环中间部分和循环后层 $b+1, \dots, N-1$ ：

$$\hat{f}(x) = \underbrace{(L_{N-1} \circ \cdots \circ L_{b+1})}_{\text{post-loop}} \circ g^{(K)} \circ \underbrace{(L_{a-1} \circ \cdots \circ L_0)}_{\text{pre-loop}}(x), \tag{2}$$

其中 $g^{(K)} : \mathbb{R}^{T\times d} \to \mathbb{R}^{T\times d}$ is a K 步迭代 g 。当 $a = 0$ 或 $b = N-1$ 时，对应的边界复合为恒等映射。映射 $g^{(K)}$ 依赖于迭代模式（第 2.2 节）和循环策略（第 2.3 节）。

2.2 块模式与层模式迭代

给定 (1) 中的窗口算子 $g = L_b \circ \cdots \circ L_a$ ， $g^{(K)}$ 有两种自然实现方式，

$$\text{block-mode:} \quad g_{\text{blk}}^{(K)}(x) = (L_b \circ \cdots \circ L_a)^K(x), \tag{3}$$

$$\text{layer-mode:} \quad g_{\text{lyr}}^{(K)}(x) = L_b^K \circ L_{b-1}^K \circ \cdots \circ L_a^K(x). \tag{4}$$

换言之，块模式将整个窗口作为一个单元进行迭代，而层模式在传递到下一层之前逐层迭代（图 1）。对于稠密模型 [4, 5]，两者产生大致相似的质量。然而，对于专家混合（MoE）层 [25, 55, 23]，块模式变得不稳定，因为在迭代 i 时，窗口中每个 MoE 层的门控网络看到略微扰动的隐藏状态，并路由到与迭代 $i-1$ 时不同的专家子集，累积的路由诱导噪声最终压倒了预期的细化。另一方面，层模式迭代只计算一次门控决策，并应用相同的专家混合 K 次 [23]，避免了这种失败模式，使其成为 MoE 骨干的正确默认选择。

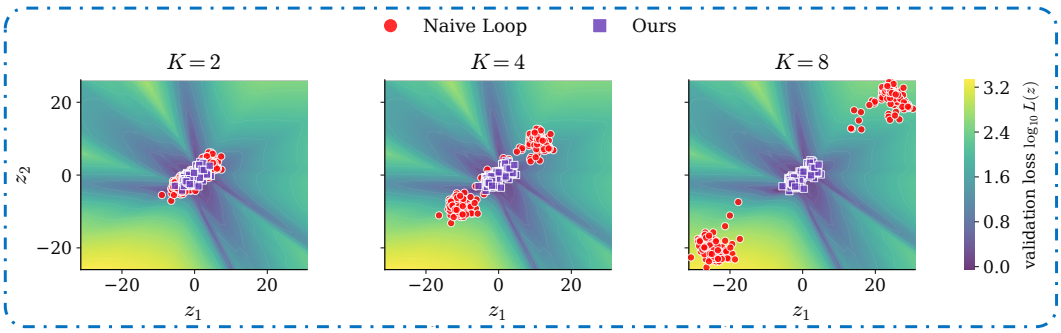


图 2：RK 积分与朴素循环。一个微小的 MLP 前（$\mathbb{R}^4 \to \mathbb{R}^2$）→ 3 个残差层块（$\mathbb{R}^2 \to \mathbb{R}^2$）→ 后（$\mathbb{R}^2 \to \mathbb{R}^2$）在二维回归目标上进行端到端训练。每个面板固定 $K \in \{2, 4, 8\}$，并在后块隐藏状态 $\mathbf{z}$ 的 220×220 网格上显示中位数测试损失 $L(\mathbf{z})$ =中位数 $_i \| 后（$\mathbf{z}) - y_i\|^2$（对数尺度，着色）以及通过朴素循环（红色圆圈）和 K 阶段 RK 积分（紫色方块）获得的测试集散点。训练后的基线诱导出一个中央低损失谷（蓝色）；我们的端点在每个 K 处都聚集在该谷中，而朴素 K 循环端点随着 K 的增加逐渐向外漂移到高损失区域（黄色）。

2.3 循环策略

我们首先通过一个结构性观察来激发几种循环策略。一个标准的预归一化变换器层 L [87] 实现为

$$L(x) = x + \mathrm{Attn}(\mathrm{LN}_1(x)) + \mathrm{MLP}(\mathrm{LN}_2(x + \mathrm{Attn}(\mathrm{LN}_1(x)))). \tag{5}$$

右侧是层的输入加上一个残差更新。将其直接推广到循环窗口算子 g，该算子可以是单层 $g = L_i$（层模式情况）或连续复合 $g = L_b \circ \cdots \circ L_a$（块模式情况），我们定义窗口残差场

$$F_g(x) := g(x) - x. \tag{6}$$

根据构造，$g(x) = x + F_g(x)$，这恰好是步长为 h=1 的前向欧拉步

$$x_1 = x_0 + h \cdot F_g(x_0) \tag{7}$$

在自治常微分方程 [6, 7] 上

$$\dot{x} = F_g(x). \tag{8}$$

无论窗口包含一层还是多层，这一点都成立，两种模式的区别仅在于 F_g 展开为何。在逐层模式下，F_g 就是单层残差场

$$F_g(x) = \mathrm{Attn}_i(\mathrm{LN}_1^i(x)) + \mathrm{MLP}_i(\mathrm{LN}_2^i(\cdot)).$$

在逐块模式下，对展开链进行叠缩（证明见附录 B）得到

$$F_g(x) = \sum_{i=a}^{b} F_{L_i}(y_i(x)), \qquad y_a(x) := x, \quad y_{i+1}(x) := y_i(x) + F_{L_i}(y_i(x)), \tag{9}$$

其中 $F_{L_i}(z) := L_i(z) - z$ 是第 i 层的层残差场。在两种模式下，循环后层 $L_{b+1}, \ldots, L_{N-1}$ 被训练为接收由一次应用 g 隐式产生的 $x(t$=1) 在 F_g 上的近似，而不是任何其他时刻的轨迹。

朴素地循环 g 共 K 轮，即 $x \leftarrow g(x)$ 被应用 K 次 [27, 47, 35, 28, 77, 95, 33, 102, 88, 1, 58, 32, 12], is a K 步前向欧拉积分，步长为 h=1, ，近似 $x(t$=$K)$。但循环后层并未被训练为接收 t=K 时刻的轨迹，且经验表明，朴素 K 倍循环几乎普遍降低性能（第 3 节）。图 2 展示了一个具有二维瓶颈的端到端可训练小网络上朴素循环与子步进的效果，其中循环后层的输入可以直接绘制。阻尼子步终点保持在训练得到的低损失谷内，而朴素 K 循环终点漂移到高损失区域。数值上，在测试集上平均时，朴素 K=2 循环使

Algorithm 3 Explicit **block-mode** **layer-mode** Runge–Kutta with Butcher tableau (13)–(15).

Input: x, range $[a, b]$, number of stages K, anchor weight $\beta \in [0, 1]$

$x \leftarrow \text{PRELOOP}(x)$ ▷ *embedding, pre-block, etc.*
$\tilde{x} \leftarrow (L_b \circ \cdots \circ L_a)(x)$ ▷ *anchor (block mode)*
for $\ell = a, \ldots, b$ **do** ▷ *outer loop over each layer*
  $\tilde{x} \leftarrow L_i(x)$ ▷ *anchor (layer mode)*
  for $k = 1, \ldots, K-1$ **do** ▷ *iterate over the K Runge–Kutta stages*
    $y \leftarrow (L_b \circ \cdots \circ L_a)(x)$ $y \leftarrow L_\ell(x)$
    $x \leftarrow x + \frac{1}{K}(y - x)$ ▷ *Runge–Kutta update*
  end for
  $x \leftarrow \beta\tilde{x} + (1-\beta)x$ ▷ *anchor combination*
end for
return $\text{POSTLOOP}(x)$

均方误差增加到 2.88，而基线和子步进值分别为 0.015 和 0.36（$\approx 8\times$ 差距）；在 K=8 时，均方误差恶化为 335，而子步进值为 1.04（$\approx 320\times$ 差距）。

因此，$g^{(K)}$ 的原则性目标不是将积分推进到 t=K，而是更好地逼近未修改网络已经瞄准的同一终点 $x(t$=$1)$。为应对这一挑战，经典数值分析建议以更细的分辨率 $h = 1/K$ 对相同的总积分时间 [0，1] 进行子步进。对 (8) 执行 K 次步长为 h=$1/K$ 的欧拉步，得到阻尼更新

$$x_{k+1} = \left(1 - \frac{1}{K}\right) x_k + \frac{1}{K} g(x_k), \tag{10}$$

该过程以 $O(1/K)$ 阶收敛至真实 $x(t$=$1)$，并严格优于网络在每一层隐式实现的单步 h=1 欧拉步。由于方程（6）–（8）与模式无关，该原理既适用于层模式（对每层场 $F_g = F_{L_i}$）进行子步进），也适用于块模式（对（9）中的复合场 F_g 进行子步进）。

更一般地，我们可以考虑超越前向欧拉法的数值积分策略来近似 $x(t$=$1)$。特别地，阻尼欧拉更新（10）可替换为 s 级显式龙格–库塔积分器

$$x_1 = x_0 + h \sum_{i=1}^{s} b_i k_i, \tag{11}$$

其中 s 各级由下式给出

$$k_1 = F_g(x_0), \qquad k_2 = F_g(x_0 + h(a_{21}k_1)), \qquad \ldots, \qquad k_s = F_g\left(x_0 + h\sum_{j=1}^{s-1} a_{sj}k_j\right), \tag{12}$$

其系数 $\{a_{ij}, b_i\}$ 由布彻表指定。算法 1 和算法 2 分别总结了该族方法在块模式和逐层模式下的实现，其中步长取为 h=1。

作为一个特别有效的示例，我们选择特定的布彻表，使龙格–库塔输出成为基础输出与（10）式 K 步阻尼欧拉更新之间的插值。具体地，我们设置且，使得 i^{th} 级在以下点处计算 F_g

$$h = 1, \quad s = K, \qquad a_{ij} = \frac{1}{K}\mathbb{I}(j < i), \tag{13}$$

$$y_i = x_0 + \frac{1}{K}\sum_{j=1}^{i-1} k_j = F^{i-1}(x_0), \qquad k_i = F_g(y_i), \tag{14}$$

表 1：常微分方程（8）数值积分的迭代策略。

Strategy	Update Rule	Forward Passes
Naive iteration & forward Euler		
Naive Loop	$x_{k+1} = g(x_k)$	K
Forward Euler [6, 7]	$x_{k+1} = x_k + \frac{1}{K} F_g(x_k)$	K
Higher-order Runge–Kutta [11]		
Midpoint (RK2)	$x_{k+1} = x_k + hF_g(x_k + \frac{h}{2} F_g(x_k))$	$2K$
Heun (RK2)	$x_{k+1} = x_k + \frac{h}{2}(F_g(x_k) + F_g(x_k + hF_g(x_k)))$	$2K$
RK4	$x_{k+1} = x_k + \frac{h}{6}(\kappa_1 + 2\kappa_2 + 2\kappa_3 + \kappa_4)$	$4K$
Fixed-point accelerators		
Heavy-ball (α, β) [73]	$x_{k+1} = x_k + \alpha F_g(x_k) + \beta(x_k - x_{k-1})$	K
Anderson (m, β) [89]	$x_{k+1} = (1-\beta)(x_k - (\Delta X_k)\gamma_k^\star) + \beta(g(x_k) - (\Delta F_k)\gamma_k^\star)$	K
Aitken Δ^2 [2]	$x_{k+1} = x_k - \frac{(d_{1,k})^2}{d_{2,k}}$ (per-coordinate)	K
Uniform Loop [57]	$x_{k+1} = g\left(\frac{1}{k+1}\sum_{i=0}^{k} x_i\right)$	K

Notation. RK4: $\kappa_1 := F_g(x_k), \kappa_2 := F_g(x_k + \frac{h}{2}\kappa_1), \kappa_3 := F_g(x_k + \frac{h}{2}\kappa_2), \kappa_4 := F_g(x_k + h\kappa_3)$. 安德森: $\gamma_k^\star := \arg\min_\gamma \|f_k - (\Delta F_k)\gamma\|_2$，其中 $f_i := g(x_i) - x_i$ 和 $\Delta X_k, \Delta F_k$ 分别为最近 m 个残差/状态增量的矩阵 [89]。艾特肯: $d_{1,k} := g(x_k) - x_k, d_{2,k} := g(g(x_k)) - 2g(x_k) + x_k$，按坐标应用；斯特芬森保护截断 $|x_{k+1} - x_k| \le |d_{1,k}|$ 并要求 K 为偶数。

$F(x) := x +$ 其中 $\frac{1}{K} F_g(x)$ 为阻尼欧拉更新（10）。然后我们选择 $\beta \in [0, 1]$ 并设置输出权重为

$$b_1 = \beta + \frac{1-\beta}{K}, \qquad b_i = \frac{1-\beta}{K}, \quad i = 2, \dots, K, \tag{15}$$

这些系数非负且和为 1。在这些系数下，龙格-库塔输出满足恒等式

$$x_1 = x_0 + \sum_{i=1}^{K} b_i k_i = \beta g(x_0) + (1-\beta) F^K(x_0), \tag{16}$$

这导致了一种高效实现（算法 3）。证明见附录 B。

算法 3 可以解释为一种龙格-库塔方法，其前置求积规则由 β. 控制。在极端情况下，$\beta = 0$ 恢复 K 子步前向欧拉端点，而 $\beta = 1$ 保留原始输出 $g(x_0)$。对于 β 的中间值，该方法通过给初始残差方向赋予更大权重，有效地使轨迹偏向训练好的单步端点。

除了龙格-库塔方法，常微分方程文献中还广泛研究了大量数值积分方案 [89, 6]。这些方法通常旨在解决底层动力学的特定性质，我们在表 1 中列出了几个突出的例子。这些算法在阶数、加速器和步长选择上有所不同，将构成我们评估的七种策略的基础。

3 实验

本节评估所提出的免训练循环包装器，涵盖稠密和专家混合检查点、基础版和指令微调版变体，以及标准多头注意力和多头潜在注意力骨干，使用固定配方且不进行逐单元超参数调优（第 3.1 节），除非另有说明。

主要发现。我们的方法在知识密集型多项选择任务上取得了最大且最可靠的收益，尤其是 MMLU-Pro、GPQA-Main 和 ARC-Challenge（第 3.4 节）。在各种架构中，大多数单元为正或中性，失败集中在某些知识型多项选择任务上的小型蒸馏检查点（第 3.5 节）。

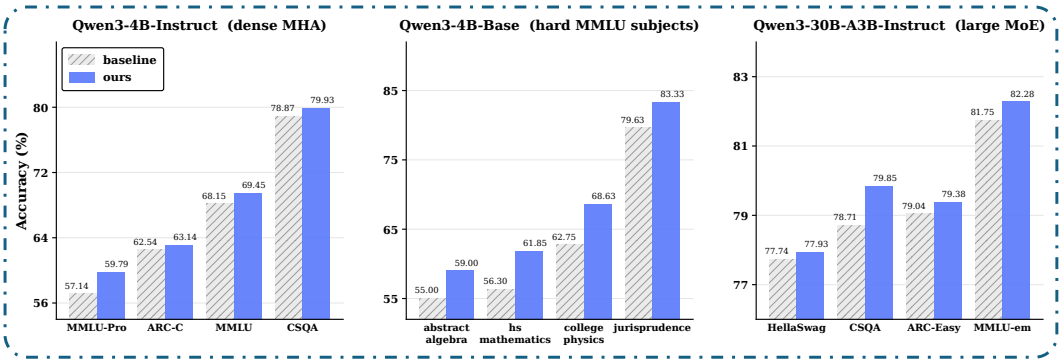


图 3：在无训练循环包装器下，三种 Qwen3 模型变体在各基准上的准确率。每个面板展示基线（灰色条纹）与本文包装器（蓝色实线）在 4 个知识型 MC 基准上的对比。每个面板的 y 轴经过裁剪以突出基线与循环之间的差距。左：Qwen3-4B-Instruct（稠密 MHA）在四个中等难度通用基准上的表现。中：Qwen3-4B-Base 在四个困难 MMLU 5-shot 科目上的表现，这些科目根据附录 H 从 MMLU 的 57 个科目中选取。右：Qwen3-30B-A3B-Instruct（大型 MoE）在四个中等难度基准上的表现。

表 2：循环配方的窗口选择。(a) 在 Qwen3-1.7B-Base 上前向欧拉法下的窗口大小扫描（K=2): n=4 是最佳点，在 $n \geq 6$. **(b)** 处出现急剧下降）。 n=4 的选择在不同模型规模和系列中具有泛化性。**(c)** 在规范循环窗口 [13，16] 下，MoE 骨干上的块模式与层模式迭代（块 K=3 /层 K=2 ）：层模式在困难 MoE 基准上带来+0.5 至+1.7 个百分点的提升。数字为 Δpp 与基线的对比。粗体标记每行获胜变体；“—”表示未运行。(a) 窗口大小扫描，Qwen3-1.7B-Base

n	Window	16-task	Δ
1	[14]	55.72	+0.18
2	[13,14]	55.60	+0.06
3	[13,14,15]	55.75	+0.21
4	[12–15]	**56.09**	**+0.55**
6	[11–16]	54.72	-0.82
12	[8–19]	54.91	-0.63
28 (all)	[0–27]	27.81	-27.73

(b) Looped layer sizes across model scales

Model	n=3 Δ	n=4 Δ	Winner
Qwen3-0.6B-Base [86]	**+0.57**	+0.22	n=3
Qwen3-1.7B-Base [86]	+0.21	**+0.55**	n=4
Qwen3-4B-Base [86]	+0.43	**+0.85**	n=4
Llama-3.2-3B-Instruct [83]	+0.23	**+0.45**	n=4

(c) Block-mode vs. layer-mode (MoE)

Benchmark	block (K=3)	layer (K=3)
Qwen1.5-MoE-A2.7B [85]		
ARC-Challenge	+0.17	**+0.85**
CSQA	+0.16	**+0.33**
OBQA	**+1.00**	-1.40
MMLU	-1.18	**0.00**
SciQ	**+0.20**	—
GPQA-Main	-1.11	—
Moonlight-16B-A3B [55]		
ARC-Challenge	-1.45	**+0.51**
CSQA	-0.66	**+0.49**
OBQA	-0.20	**+1.20**
MMLU	**+0.74**	—
SciQ	**-0.50**	-0.60
GPQA-Main	-2.00	**+0.90**

鲁棒性。有效循环窗口遵循稳定的深度比例规则，而 MoE 模型需要层模式迭代以避免路由抖动（第 3.2 节，表 2）。大量消融研究证实了对循环策略、循环次数、窗口位置、缓存处理和解码设置的鲁棒性（第 3.6 节）。

比较与迁移。与朴素的推理时循环变换器相比，本文方法避免了崩溃，并在所有测试单元中表现最佳（第 3.7 节，图 5）。无泄漏迁移至 Qwen3-30B-A3B-Instruct 进一步支持跨架构泛化（表 4）。

3.1 实验设置

我们评估了七个 Hugging Face 检查点系列，涵盖稠密和 MoE 骨干、基础版和指令微调版，以及标准 MHA 与多头潜在注意力（MLA）：Qwen3 稠密 $\{0.6, 1.7, 4\}$ B（基础版与指令版）[86]、Qwen3-MoE 30B-A3B-Instruct(qwen3_moe)[86] 、 Qwen1.5-MoE-A2.7B-Chat(qwen2_moe, 蒸馏版、Llama-3.2 $\{1, 3\}$ B-Instruct（从 Llama-3.1 蒸馏）

表 3：知识密集型多项选择基准，循环包装器直接应用于每个冻结检查点。未进行逐单元超参数调优。Δpp 表示在相同提示下相对于无循环基线的改进。逐单元配置列于附录 D，针对 Qwen3-30B-A3B-Instruct 的完全无泄漏单一配方泛化检查报告于表 4。

(a) Dense backbones

Model	Benchmark	Base	Loop (Ours)	Δpp
Qwen3-4B-Instruct [86]	MMLU-Pro 5-shot [91]	0.5714	**0.5979**	**+2.64**
	GPQA-Main 0-shot [76]	0.3371	**0.3571**	**+2.01**
	CommonsenseQA 7-shot [81]	0.7887	0.7993	+1.06
Llama-3.2-3B-Instruct [83]	GPQA-Main 0-shot [76]	0.2991	0.3103	+1.12
	MMLU-Pro 5-shot [91]	0.3164	0.3236	+0.71
	MMLU 0-shot [40]	0.5966	0.6039	+0.72
Llama-3.2-1B-Instruct [83]	GPQA-Main 0-shot [76]	0.2790	0.2969	+1.79

(b) MoE backbones

Model	Benchmark	Base	Loop (Ours)	Δpp
Qwen1.5-MoE-A2.7B [85]	ARC-Challenge 25-shot [21]	0.4829	**0.5060**	**+2.30**
	CommonsenseQA 7-shot [81]	0.7961	0.8133	+1.72
	OpenBookQA 0-shot [62]	0.3160	0.3320	+1.60
Moonlight-16B-A3B [55]	MMLU 0-shot [40]	0.6786	0.6860	+0.74
	OpenBookQA 0-shot [62]	0.3160	0.3280	+1.20
DeepSeek-V2-Lite-Chat [25]	ARC-Challenge 25-shot [21]	0.5794	0.5879	+0.85

8B/70B) [83]、DeepSeek-V2-Lite-Chat（多头潜在注意力 + 64 专家混合专家模型）[25]，以及月光 -16B-A3B-指令 (deepseek_v3 系列，多头潜在注意力 + 混合专家模型）[55]。

所有评估均使用 lm-eval-harness 0.4.11 [30]，采用 bfloat16 权重和 SDPA 注意力。对于每个（模型，基准）单元，我们应用单一开箱即用方案：在中间 4 层使用 3 阶段 Runge – Kutta，密集骨干使用块模式，MoE 骨干使用层模式，且不对位置、 K 或策略进行逐单元超参数搜索。绝对层索引和次要设置（缓存、解码）的逐单元变化机械地源于各架构的层数，并记录在附录 D 中以供复现；将该方案完全无泄漏地迁移到留出架构（Qwen3-30B-A3B-Instruct），且无任何逐单元变化，报告于

表 4。

3.2 深度分数规则

在所有八种测试架构中，最优窗口的中心位于分数深度的窄带内（表 2，并在附录 Q 的图 6 中跨九种架构可视化）。对于大于 1.7B 的模型，最优值位于上半部分（0.43 – 0.71，众数 ≈ 0.50）；对于小于 1B 的模型，则提前（0.25 – 0.56）。我们假设头部特化集中在后期层，循环必须避开这些层 [10, 46, 59]；在小型或高度蒸馏的模型中，后期层占比更大，因此安全窗口更早开始 [80, 75]。

3.3 混合专家模型的分层模式

在混合专家模型（MoE）主干网络 [25, 55] 上，默认的块模式迭代会导致路由抖动，即每次块迭代都会在轻微扰动的状态上重新评估门控函数，从而累积由路由引起而非表示引起的变化。如第 2.2 节和 [23, 4] 所述，可通过切换到层模式迭代来解决此问题。表 2 比较了两种模式在 Qwen1.5-MoE 和 Moonlight 上的表现，可以看出层模式将大多数负值单元格翻转为正值，在困难的混合专家模型基准上带来+0.5 至+1.7 个百分点的提升。

3.4 基于知识的 MC 任务结果

我们的主要实验结果涉及知识密集型多项选择任务，其中基线低于上限，并使用宽松提取器进行评估。在这类单元上，循环充当冻结上下文

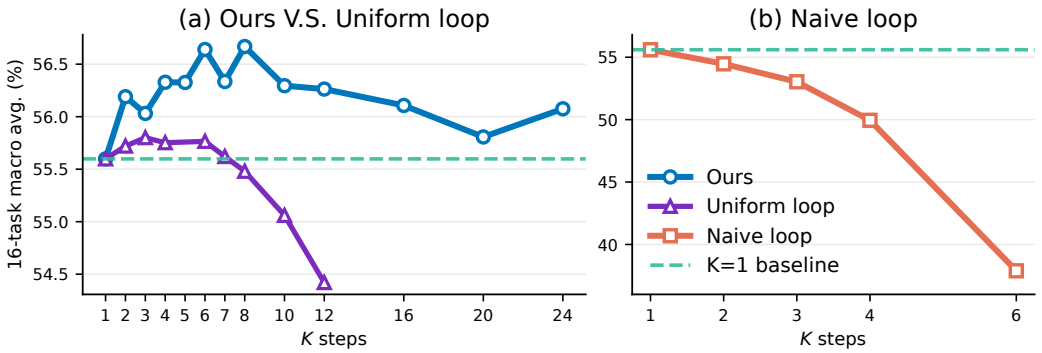


图 4：循环次数 K 对 16 任务宏平均准确率的影响。(a) 我们的方法（算法 5）在 $K \in \{1, 2, \cdots, 24\}$ 范围内保持稳定，而均匀循环 [57]（表 1）在 $K \geq 6$. **(b)** 之后无法扩展。朴素循环变体单调退化，在 $K=6$ 处降至 37.89%（-17.71 个百分点）。

表 4：Qwen3-30B-A3B-Instruct 在 [22，24] 处的泛化检查。配置根据第 3.2 节的深度分数规则预先固定，没有逐单元搜索或调优，即配置在任何单元评分之前就已确定，因此每个单元都是完全无泄漏的迁移（附录 K）。粗体标记两个已提交求解器选择中较好的一个（算法 3 中的 $K=2$ 阶段龙格-库塔法与表 8 中定义的 Heun $K=1$ 法）。

	Accuracy / exact-match (↑)								PPL (↓)
Method	ARC-Easy	HellaSwag	SciQ	CSQA	TruthfulQA	MMLU-flex	GPQA-Main	SuperGPQA	LAMBADA
Baseline	79.04	77.74	94.80	78.71	34.15	66.67	37.28	31.00	4.12
K-stage RK	**79.38**	**77.93**	**95.00**	**79.85**	**34.64**	**67.46**	37.50	**31.70**	**4.11**
Δ vs. baseline	+0.34	+0.19	+0.20	+1.14	+0.49	+0.79	+0.22	+0.70	-0.01
Heun $K=1$	79.08	77.78	94.90	79.61	34.27	66.67	**37.72**	31.05	4.12
Δ vs. baseline	+0.04	+0.04	+0.10	+0.90	+0.12	0.00	+0.44	+0.05	0.00

知识精炼器，且增益最大且最稳健。图 3 和表 3 报告了每个模型家族的最佳循环配置，分为稠密和 MoE 骨干。

最大增益（+2.0 至+2.6 个百分点）出现在最强 MHA 骨干的最难任务上（MMLU-Pro [91]、GPQA-Main [76]、ARC-Challenge [21]）。基于 MLA 的 MoE 模型（DeepSeek-V2-Lite、Moonlight）方向相同，但幅度小 3 – 4 × 。

3.5 跨架构总结

在七个模型家族和统一的知识 MC 任务套件中，我们按照第 3.1 节的固定配方对每个单元进行评分，得到 27 个正单元（$\Delta > +0.3$ 个百分点，60%）和 12 个中性单元（$|\Delta| \leq 0.3$ 个百分点，27%）。其余负单元集中在一个单一区域，即知识 MC 上的 3B 以下蒸馏检查点（例如 Llama-3.2-1B 在多项选择 MMLU [40] 上）。即使在该区域，GPQA-Main [76] 在 Llama-3.2-1B 上确实在非常早的位置 [4，7] 转为正值（+1.79），表明失败边界是任务相关的而非绝对的。该包装器在稠密、MoE、基础、指令和蒸馏检查点上 87% 的单元产生正或中性信号。

3.6 消融研究

我们系统性地进行了大量消融研究，完整表格见附录。这些研究包括集成策略与阻尼调度（附录 E）、循环次数 K（图 4）、窗口宽度与位置（附录 G 和 M）、块模式与层模式迭代（附录 L）、键值缓存与解码时处理（附录 N 和 I），以及改变窗口位置、策略、迭代次数和缓存选择的配方鲁棒性检查（附录 D 和 P）。我们进一步在附录 J、H、K 和 G 中报告了鲁棒性检查、逐主题分解、大规模未调优迁移以及失败配置分析。

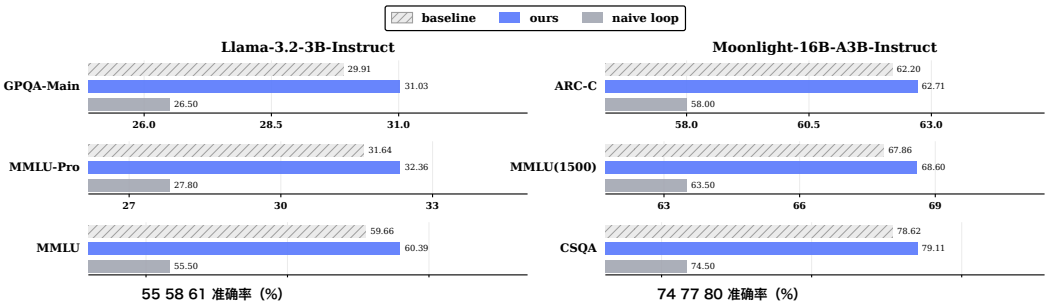


图 5：在 Llama-3.2-3B-Instruct 和 **Moonlight-16B-A3B-Instruct** 上与其他循环变换器方法的比较。在每个骨干网络上，我们报告了三种配置下的三个知识多项选择基准：基线（无循环，原始检查点）、带 $K=4$ 的朴素循环，以及我们的方法（算法 3）。

3.7 与其他循环变换器方法的比较

我们进一步通过在同一冻结检查点 [27, 28, 33, 102, 4, 5, 42, 67, 82, 77, 95, 94] 上与两种自然替代方案进行比较，来分离我们方法的贡献。[35] 的朴素循环变换器在每个单元上均崩溃，因为迭代离开了循环后层所训练的状态域（第 2.3 节），准确率下降了几个百分点。相比之下，我们的方法保持在训练状态域内，在几乎每个单元上都取得了最高准确率。图 5 报告了在 Llama-3.2-3B-Instruct（GPQA-Main [76]、MMLU-Pro [91]、MMLU [40]）和 Moonlight-16B-A3B-Instruct（ARC-C [21]、MMLU、CSQA [81]）上的结果。

4 相关工作

循环与递归深度变换器。大量工作将递归引入训练阶段的变换器，从权重绑定堆叠 [27, 47] 和循环表达能力结果 [35, 95, 61, 36, 97, 34]，到长度泛化研究 [28, 24, 78, 9] 和大规模循环预训练 [33, 102, 74, 82, 94]。自适应递归变体 [31, 23, 5, 4, 66] 和隐式深度不动点 [6, 7] 同样将递归与训练绑定，最近的弹性深度潜在推理模型 [42, 67, 14, 43, 98, 70, 45] 也是如此。我们的包装器则针对未修改的已发布检查点 [86, 83, 25, 55]，无需训练 [51, 54, 50, 64, 15, 16, 65]、无辅助损失、无架构改动。

推理时计算与潜在推理。另一条轴线将测试时计算用于更长的轨迹，例如思维链 [92, 63]、自洽性 [90] 和经过推理训练的语言模型 [26]，这些方法均保持逐令牌前向传播不变；理论刻画了它们的优势 [29, 60, 96, 77]。潜在推理路线则将计算推入连续空间 [39, 20, 72, 37, 101, 99, 33, 48]，或使用自适应停止来“思考”[38, 8, 44, 56, 49, 13]，呼应了将前向计算视为迭代精化的调谐透镜观点 [10] 和层冗余发现 [59, 46, 80, 75]。我们的干预是正交的，因为我们在冻结检查点的单次前向传播中为每个令牌使用更多计算。

数值分析方法。将残差块视为前向欧拉步，激发了我们的包装器在循环窗口上暴露的积分策略族，这与不动点图景 [6, 7] 一致。在该族中，我们基准测试了通过安德森加速 [89]、重球法 [73, 19, 17, 71, 52, 79]、艾特肯外推和高阶龙格-库塔求解器实现的皮卡德加速，发现没有一种方法能稳健地优于 K 阶龙格-库塔，这表明预训练变换器 [35, 77, 5] 的循环中间部分在任何有用意义上都不是收缩的 [6, 10]。并行的免训练方向通过层跳过 [59, 46] 操纵深度，但据我们所知，没有先前方法在现代检查点 [86, 83, 25, 55] 上结合中间块循环和层模式迭代用于专家混合路由 [18]。

5 结论

我们的免训练包装器适用于冻结的已发布检查点（Qwen3、Llama-3.2、DeepSeek-V2-Lite、Moonlight、Qwen1.5-MoE），通过将每个预归一化变换器块重新解释为在 $h{=}1$ 处的前向欧拉步骤，并通过迭代连续循环窗口 K 次来提供更好的近似。块模式和层模式迭代均受支持，其中层模式在 MoE 骨干上必需，以稳定每层路由。通用深度分数规则将最优窗口置于分数深度 $\approx 0.45 - 0.60$ 之间，适用于从 16 到 48 层的稠密和 MoE 架构。增益集中在知识密集型多项选择基准（MMLU-Pro [91]、GPQA-Main [76]、ARC-Challenge [21]）上，我们的方法在这些基准上增加了约 +2 个百分点，并且在七个模型家族和我们的基准套件中，87% 的单元格在固定的开箱即用配方下为非负值，且无需逐单元格超参数调优。

参考文献

[1] Kwangjun Ahn, Xiang Cheng, Hadi Daneshmand, and Suvrit Sra. Transformers learn to implement preconditioned gradient descent for in-context learning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, 2023.

[2] Alexander C. Aitken. On Bernoulli's numerical solution of algebraic equations. *Proceedings of the Royal Society of Edinburgh*, 46:289–305, 1927.

[3] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *CoRR*, abs/2108.07732, 2021.

[4] Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise lora. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.

[5] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron C. Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *Advances in Neural Information Processing Systems 39: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025*, 2025.

[6] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019*, pages 688–699, 2019.

[7] Shaojie Bai, Vladlen Koltun, and J. Zico Kolter. Multiscale deep equilibrium models. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020*, 2020.

[8] Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. *CoRR*, abs/2107.05407, 2021.

[9] Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. End-to-end algorithm synthesis with recurrent networks: Extrapolation without overthinking. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022*, 2022.

[10] Nora Belrose, Zach Furman, Logan Smith, Danny Halawi, Igor Ostrovsky, Lev McKinney, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *CoRR*, abs/2303.08112, 2023.

[11] John C. Butcher. *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons, 3rd edition, 2016.

[12] Bo Chen, Xiaoyu Li, Yingyu Liang, Zhenmei Shi, and Zhao Song. Bypassing the exponential dependency: Looped transformers efficiently learn in-context by multi-step gradient descent. In *International Conference on Artificial Intelligence and Statistics, AISTATS 2025*, Proceedings of Machine Learning Research, pages 4447–4455, 2025.

[13] Guanxu Chen, Dongrui Liu, and Jing Shao. Loop as a bridge: Can looped transformers truly link representation space and natural language outputs? *CoRR*, abs/2601.10242, 2026.

[14] Hung-Hsuan Chen. Thinking deeper, not longer: Depth-recurrent transformers for compositional generalization. *CoRR*, abs/2603.21676, 2026.

[15] Lizhang Chen. Demystifying LION: a Hamiltonian perspective. Master's thesis, The University of Texas at Austin, 2025.

[16] Lizhang Chen, Jonathan Li, Kaizhao Liang, Baiyu Su, Cong Xie, Nuo Wang Pierse, Chen Liang, Ni Lao, and Qiang Liu. Cautious weight decay. In *The Fourteenth International Conference on Learning Representations, ICLR 2026*, 2026.

[17] Lizhang Chen, Jonathan Li, and Qiang Liu. Muon optimizes under spectral norm constraints. *Trans. Mach. Learn. Res.*, 2026, 2026.

[18] Lizhang Chen, Jonathan Li, Qi Wang, Runlong Liao, Shuozhe Li, Chen Liang, Ni Lao, and Qiang Liu. ϕ-balancing for mixture-of-experts training. In *Forty-third International Conference on Machine Learning, ICML 2026*, 2026.

[19] Lizhang Chen, Bo Liu, Kaizhao Liang, and Qiang Liu. Lion secretly solves a constrained optimization: As Lyapunov predicts. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.

[20] Jeffrey Cheng and Benjamin Van Durme. Compressed chain of thought: Efficient reasoning through dense representations. *CoRR*, abs/2412.13171, 2024.

[21] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. *CoRR*, abs/1803.05457, 2018.

[22] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021.

[23] Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D. Manning. Moeut: Mixture-of-experts universal transformers. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*, 2024.

[24] Artur Back de Luca and Kimon Fountoulakis. Simulation of graph algorithms with looped transformers. In *Forty-first International Conference on Machine Learning, ICML 2024*, Proceedings of Machine Learning Research, pages 2319–2363, 2024.

[25] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *CoRR*, abs/2405.04434, 2024.

[26] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025.

[27] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *7th International Conference on Learning Representations, ICLR 2019*, 2019.

[28] Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped transformers for length generalization. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.

[29] Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, 2023.

[30] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 2023.

[31] Yihang Gao, Chuanyang Zheng, Enze Xie, Han Shi, Tianyang Hu, Yu Li, Michael Ng, Zhenguo Li, and Zhaoqiang Liu. Algoformer: An efficient transformer framework with algorithmic structures. *Trans. Mach. Learn. Res.*, 2025, 2025.

[32] Khashayar Gatmiry, Nikunj Saunshi, Sashank J. Reddi, Stefanie Jegelka, and Sanjiv Kumar. Can looped transformers learn to implement multi-step gradient descent for in-context learning? In *Forty-first International Conference on Machine Learning, ICML 2024*, Proceedings of Machine Learning Research, pages 15130–15152, 2024.

[33] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *CoRR*, abs/2502.05171, 2025.

[34] Jonas Geiping, Xinyu Yang, and Guinan Su. Efficient parallel samplers for recurrent-depth models and their connection to diffusion language models. *CoRR*, abs/2510.14961, 2025.

[35] Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning, ICML 2023*, Proceedings of Machine Learning Research, pages 11398–11442, 2023.

[36] Zixuan Gong, Jiaye Teng, and Yong Liu. What makes looped transformers perform better than non-recursive ones (provably). *CoRR*, abs/2510.10089, 2025.

[37] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.

[38] Alex Graves. Adaptive computation time for recurrent neural networks. *CoRR*, abs/1603.08983, 2016.

[39] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *CoRR*, abs/2412.06769, 2024.

[40] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *9th International Conference on Learning Representations, ICLR 2021*, 2021.

[41] Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Yao Fu, Maosong Sun, and Junxian He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, 2023.

[42] Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. Loopformer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *The Fourteenth International Conference on Learning Representations, ICLR 2026*, 2026.

[43] Jonas Knupp, Jan Hendrik Metzen, Jeremias Bohn, Georg Groh, and Kristian Kersting. Depth-recurrent attention mixtures: Giving latent reasoning the attention it deserves. *CoRR*, abs/2601.21582, 2026.

[44] Harsh Kohli, Srinivasan Parthasarathy, Huan Sun, and Yuekun Yao. Loop, think, & generalize: Implicit reasoning in recurrent-depth transformers. *CoRR*, abs/2604.07822, 2026.

[45] Yeskendir Koishekenov, Aldo Lipani, and Nicola Cancedda. Encode, think, decode: Scaling test-time reasoning with recursive latent thoughts. *CoRR*, abs/2510.07358, 2025.

[46] Vedang Lad, Wes Gurnee, and Max Tegmark. The remarkable robustness of llms: Stages of inference? *CoRR*, abs/2406.19384, 2024.

[47] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *8th International Conference on Learning Representations, ICLR 2020*, 2020.

[48] Shuozhe Li, Vaishnav Tadiparthi, Kwonjoon Lee, Nakul Agarwal, Hossein Nourkhiz Mahjoub, Ehsan Moradi-Pari, Lizhang Chen, Amy Zhang, and Liu Leqi. Learning robust reasoning through guided adversarial self-play. *CoRR*, abs/2602.00173, 2026.

[49] Jia Liang and Liangming Pan. Do latent-cot models think step-by-step? A mechanistic study on sequential reasoning tasks. *CoRR*, abs/2602.00449, 2026.

[50] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code. In *The Fourteenth International Conference on Learning Representations, ICLR 2026*, 2026.

[51] Kaizhao Liang, Bo Liu, Lizhang Chen, and Qiang Liu. Memory-efficient LLM training with online subspace descent. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*, 2024.

[52] Runlong Liao, Jian Yu, Baiyu Su, Chi Zhang, Lizhang Chen, and Qiang Liu. Momentum guidance: Plug-and-play guidance for flow models. *CoRR*, abs/2602.20360, 2026.

[53] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022*, pages 3214–3252, 2022.

[54] Bo Liu, Lemeng Wu, Lizhang Chen, Kaizhao Liang, Jiaxu Zhu, Chen Liang, Raghuraman Krishnamoorthi, and Qiang Liu. Communication efficient distributed training with distributed Lion. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*, 2024.

[55] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for LLM training. *CoRR*, abs/2502.16982, 2025.

[56] Wenquan Lu, Yuechuan Yang, Kyle Lee, Yanshu Li, and Enqi Liu. Latent chain-of-thought? decoding the depth-recurrent transformer. *CoRR*, abs/2507.02199, 2025.

[57] Jonathan Lys, Vincent Gripon, Bastien Pasdeloup, Axel Marmoret, Lukas Mauch, Fabien Cardinaux, and Ghouthi Boukli Hacene. Inner loop inference for pretrained transformers: Unlocking latent capabilities without training. *CoRR*, abs/2602.14759, 2026.

[58] Arvind V. Mahankali, Tatsunori Hashimoto, and Tengyu Ma. One step of gradient descent is provably the optimal in-context learner with one layer of linear self-attention. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.

[59] Xin Men, Mingyu Xu, Qingyu Zhang, Qianhao Yuan, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. In *Findings of the Association for Computational Linguistics, ACL 2025*, Findings of ACL, pages 20192–20204, 2025.

[60] William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.

[61] William Merrill and Ashish Sabharwal. A little depth goes a long way: The expressive power of log-depth transformers. *CoRR*, abs/2503.03961, 2025.

[62] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? A new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, 2018.

[63] Amirkeivan Mohtashami, Matteo Pagliardini, and Martin Jaggi. Cotformer: A chain of thought driven architecture with budget-adaptive computation cost at inference. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.

[64] Son Nguyen, Lizhang Chen, Bo Liu, and Qiang Liu. Memory-efficient optimization with factorized Hamiltonian descent. In *International Conference on Artificial Intelligence and Statistics, AISTATS 2025*, Proceedings of Machine Learning Research, pages 2863–2871, 2025.

[65] Son Nguyen, Bo Liu, Lizhang Chen, and Qiang Liu. Improving adaptive moment optimization via preconditioner diagonalization. In *International Conference on Artificial Intelligence and Statistics, AISTATS 2026*, 2026.

[66] Mohammadmahdi Nouriborji, Morteza Rohanian, and Omid Rohanian. Improving recursive transformers with mixture of loras. *CoRR*, abs/2512.12880, 2025.

[67] Costin-Andrei Oncescu, Depen Morwani, Samy Jelassi, Alexandru Meterez, Mujin Kwun, and Sham M. Kakade. The recurrent transformer: Greater effective depth and efficient decoding. *CoRR*, abs/2604.21215, 2026.

[68] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on Health, Inference, and Learning, CHIL 2022*, Proceedings of Machine Learning Research, pages 248–260, 2022.

[69] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, Volume 1: Long Papers*, 2016.

[70] Francesco Pappone, Donato Crisostomi, and Emanuele Rodolà. Two-scale latent dynamics for recurrent-depth transformers. *CoRR*, abs/2509.23314, 2025.

[71] Bowen Peng, Lizhang Chen, Baiyu Su, Jeffrey Quesnelle, Diederik P. Kingma, and Qiang Liu. DeMo: Decoupled momentum optimization. In *The Fourteenth International Conference on Learning Representations, ICLR 2026*, 2026.

[72] Jacob Pfau, William Merrill, and Samuel R. Bowman. Let's think dot by dot: Hidden computation in transformer language models. *CoRR*, abs/2404.15758, 2024.

[73] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[74] Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling laws for stable looped language models. *CoRR*, abs/2604.12946, 2026.

[75] Machel Reid, Edison Marrese-Taylor, and Yutaka Matsuo. Subformer: Exploring weight sharing for parameter efficiency in generative transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, Findings of ACL, pages 4081–4090, 2021.

[76] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level Google-proof Q&A benchmark. In *First Conference on Language Modeling*, 2024.

[77] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.

[78] Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Furong Huang, Uzi Vishkin, Micah Goldblum, and Tom Goldstein. Can you learn an algorithm? generalizing from easy to hard problems with recurrent networks. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021*, pages 6695–6706, 2021.

[79] Baiyu Su, Lizhang Chen, and Qiang Liu. The curious case of AdamW, 2026.

[80] Sho Takase and Shun Kiyono. Lessons on parameter sharing across layers in transformers. In *Proceedings of The Fourth Workshop on Simple and Efficient Natural Language Processing, SustaiNLP 2023*, pages 78–90, 2023.

[81] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. Commonsenseqa: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Volume 1 (Long and Short Papers)*, pages 4149–4158, 2019.

[82] Guo Tang, Shixin Jiang, Heng Chang, Nuo Chen, Yuhan Li, Huiming Fan, Jia Li, Ming Liu, and Bing Qin. Looprpt: Reinforcement pre-training for looped language models. *CoRR*, abs/2603.19714, 2026.

[83] Llama Team. The llama 3 herd of models. *CoRR*, abs/2407.21783, 2024.

[84] M-A-P Team. Supergpqa: Scaling LLM evaluation across 285 graduate disciplines. *CoRR*, abs/2502.14739, 2025.

[85] Qwen Team. Qwen1.5-MoE: Matching 7b model performance with 1/3 activated parameters. Technical report, Alibaba Group, 2024.

[86] Qwen Team. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025.

[87] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, pages 5998–6008, 2017.

[88] Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning, ICML 2023*, Proceedings of Machine Learning Research, pages 35151–35174, 2023.

[89] Homer F. Walker and Peng Ni. Anderson acceleration for fixed-point iterations. *SIAM J. Numer. Anal.*, 49(4):1715–1735, 2011.

[90] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023*, 2023.

[91] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*, 2024.

[92] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022*, 2022.

[93] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text, NUT@EMNLP 2017*, pages 94–106, 2017.

[94] Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel loop transformer for efficient test-time computation scaling. *CoRR*, abs/2510.24824, 2025.

[95] Kevin Xu and Issei Sato. On expressive power of looped transformers: Theoretical analysis and enhancement via timestep encoding. In *Forty-second International Conference on Machine Learning, ICML 2025*, Proceedings of Machine Learning Research, 2025.

[96] Kevin Xu and Issei Sato. To cot or to loop? A formal comparison between chain-of-thought and looped transformers. *CoRR*, abs/2505.19245, 2025.

[97] Liu Yang, Kangwook Lee, Robert D. Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.

[98] Chengting Yu, Xiaobo Shu, Yadao Wang, Yizhen Zhang, Haoyi Wu, You Wu, Rujiao Long, Ziheng Chen, Yuchi Xu, Wenbo Su, and Bo Zheng. Spiralformer: Looped transformers can learn hierarchical dependencies via multi-resolution recursion. *CoRR*, abs/2602.11698, 2026.

[99] Qifan Yu, Zhenyu He, Sijie Li, Zhou Xun, Jun Zhang, Jingjing Xu, and Di He. Enhancing auto-regressive chain-of-thought through loop-aligned reasoning. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2026 - Volume 1: Long Papers*, pages 2206–2222, 2026.

[100] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Volume 1: Long Papers*, pages 4791–4800, 2019.

[101] Boyi Zeng, Shixiang Song, Siyuan Huang, Yixuan Wang, He Li, Ziwei He, Xinbing Wang, Zhiyu Li, and Zhouhan Lin. Pretraining language models to ponder in continuous space. *CoRR*, abs/2505.20674, 2025.

[102] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models. *CoRR*, abs/2510.25741, 2025.

附录

A 符号与术语表 20

B 证明 20

B.1 (9) 的证明 20

B.2 (16) 的证明 21

C 解码循环算法 21

C.1 解码时循环前向传播 21

C.2 快照/恢复 21

C.3 decode_mode 变体 22

D 逐单元配置 22

D.1 逐单元表中可见的模式 23

E 策略消融表 23 E.1 Anderson、heavy-ball、Aitken、 α -调度 . . . . . 23 E.2 范数稳定与多项式混合 . . . . . 24 E.3 层模式复现结果 25

F 计算与可复现性 26

F.1 软件栈 26

F.2 每模型资源占用 26

G 失败配置日志 26 G.1 朴素 $K{=}4$ 循环上的灾难性崩溃 . . . . . 27 G.2 cache=none 在解码上一致地灾难性 . . . . . 27 G.3 宽循环窗口爆炸 . . . . . 27 G.4 高阶常微分方程求解器全部退化 . . . . . 27 G.5 位置搜索对 16 任务聚合过拟合 . . . . . 28 G.6 层模式 $K{=}3$ 一致地灾难性 28

G.7 在 $K{=}8$ . . . . . 处的 Anderson 28

G.8 低于 10 亿参数的知识多项选择 28

H 基准增益的集中度 28

I 免训练循环的墙钟成本 29

J 鲁棒性检查 29

J.1 留出验证纪律 29

J.2 Qwen3-4B-Instruct 上各配置的鲁棒性 30

J.3 小模型上的多个获胜窗口 30

J.4 语言建模困惑度得以保持 . . . . . 30 J.5 效应方向在不同少样本数量下保持一致 30

K 扩展至 30B：Qwen3-30B-A3B-Instruct 更广泛扫描 30

L 超越 MoE 骨干的层级模式优势 31

M 密集 Qwen3 尺寸下的损失面广度 31

N Qwen3-4B-Instruct 上缓存策略的鲁棒性 32

N.1 KV 缓存处理 32

O 各架构实现说明 32

P 超参数搜索协议 33

Q 九种架构的深度比例规则 34

A 符号与术语表

表 5 汇总了全文及附录中使用的符号。

表 5：符号、类型及其引入位置。

Symbol	Type / range	Meaning
Network		
N	$\mathbb{N}_{>0}$	Number of decoder layers in the released checkpoint.
L_i	$\mathbb{R}^{T\times d}\to\mathbb{R}^{T\times d}$	Pre-norm transformer block i, $0 \le i \le N-1$.
f	composition	Unmodified network: $f = L_{N-1}\circ\cdots\circ L_0$.
T, d	$\mathbb{N}_{>0}$	Sequence length and hidden dimension.
Loop wrapper		
$[a, b]$	$0 \le a \le b \le N-1$	Loop-window layer indices (contiguous).
W	$b-a+1$	Loop window width (in layers).
K	$\mathbb{N}_{\ge 1}$	Loop iteration count.
g	$\mathbb{R}^{T\times d}\to\mathbb{R}^{T\times d}$	Block operator: $g = L_b\circ\cdots\circ L_a$ (Eq. 1).
$g^{(K)}$	$\mathbb{R}^{T\times d}\to\mathbb{R}^{T\times d}$	K-step iteration of g under the chosen strategy (Section 2.3).
$\hat{f}(x)$	$\mathbb{R}^{T\times d}$	Looped network output (2).
$g^{(K)}_{\mathrm{blk}}, g^{(K)}_{\mathrm{lyr}}$	maps	Block-mode and layer-mode realizations of $g^{(K)}$ (3)–(4).
ODE view		
F_g	$\mathbb{R}^{T\times d}\to\mathbb{R}^{T\times d}$	Window residual field $F_g(x) := g(x) - x$ (6).
F_{L_i}	$\mathbb{R}^{T\times d}\to\mathbb{R}^{T\times d}$	Single-layer residual field $L_i(x) - x$.
$y_i(x)$	$\mathbb{R}^{T\times d}$	Intra-window residual stream at layer i (9).
h	$\mathbb{R}_{>0}$	Forward-Euler step size; h=1 is the layer-built-in step.
α	$(0, 1]$	Damped-Euler coefficient α; $\alpha = h$.
$\kappa_1, \ldots, \kappa_4$	$\mathbb{R}^{T\times d}$	RK4 stage evaluations (Table 1).
m, β	$\mathbb{N}$, $\mathbb{R}$	Anderson memory depth and mixing parameter.
KV cache		
$\mathcal{C}_i$	cache slot	Per-layer KV slot for layer i.
ℓ_i	$\mathbb{N}$	Snapshot length of $\mathcal{C}_i$ before a loop iteration.
c	{FIRST, LAST, NONE}	Cache strategy (19).

B 证明

B.1 公式 (9) 的证明

我们将块模式迭代展开为逐层前向欧拉步骤的显式链，并验证第 2.3 节所述的闭式恒等式。

设 $g = L_b \circ L_{b-1} \circ \cdots \circ L_a$ 为循环窗口算子。每个组成层满足单层前向欧拉恒等式 $L_i(z) = z + F_i(z)$。固定输入 x。令 $y_a(x) := x$，并递归定义窗口内残差流为

$$y_{i+1}(x) := L_i(y_i(x)) := y_i(x) + F_i\big(y_i(x)\big), \qquad i = a, a+1, \ldots, b. \tag{17}$$

根据构造，$g(x) = y_{b+1}(x)$。将 (17) 从 $i = a$ 到 $i = b$ 求和，得到

$$\sum_{i=a}^{b}\big(y_{i+1}(x) - y_i(x)\big) = \sum_{i=a}^{b} F_i\big(y_i(x)\big).$$

左侧望远镜求和为 $y_{b+1}(x) - y_a(x) = g(x) - x$。因此

$$g(x) - x = \sum_{i=a}^{b} F_i(y_i(x)) =: F_g(x), \tag{18}$$

这恰好是 (9)。

B.2 (16) 的证明

根据 (14)，标准 K 子步前向欧拉端点可写为

$$F^{(K)}(x_0) = x_0 + \frac{1}{K}\sum_{i=1}^{K} k_i.$$

代入 (16) 的右侧，我们有

$$\begin{aligned}\beta g(x_0) + (1-\beta)F^{(k)}(x_0) &= \beta g(x_0) + (1-\beta)x_0 + \frac{1-\beta}{K}\sum_{i=1}^{K} k_i \\ &= \beta g(x_0) + (1-\beta)x_0 + \frac{1-\beta}{K}F_g(x_0) + \frac{1-\beta}{K}\sum_{i=2}^{K} k_i \\ &= x_0 + \left(\beta + \frac{1-\beta}{K}\right)k_1 + \frac{1-\beta}{K}\sum_{i=2}^{K} k_i,\end{aligned}$$

其中第二行和第三行使用了 $k_1 = F_g(x_0) = g(x_0) - x_0$ 。利用 (15) 中的定义展开 (16) 的左侧后，结论随之成立。

C 解码循环算法

我们修补到冻结模型中的循环前向传播包含两种状态，它们共享大部分代码，但在循环体与键值缓存的交互方式上有所不同。在预填充阶段 (seq_len $>$ 1，没有过去的键值缓存），循环体以 use_cache=False 运行，并且不向缓存写入任何内容；只有在循环之后的一次暂存传递才会写入规范的键值。在解码阶段 (seq_len == 1，存在过去的键值缓存），循环体必须关注过去的键值缓存；否则，新标记将基于截断的上下文进行计算。然而，它也必须不产生任何净键值写入，因为暂存传递随后会写入规范条目。其机制是在每次循环迭代之前快照每个循环层的缓存长度，并在迭代之后立即裁剪回这些长度。

C.1 解码时循环前向传播

算法 4 将完整的块模式解码时前向传播表述为一个单一过程，包括预循环/循环体/暂存/后循环阶段、每次迭代的快照/恢复协议以及缓存策略分支。算法 5 给出了层模式的对应过程，其区别仅在于迭代和快照/恢复的嵌套方式。

算法 4 的第 4 – 5 行记录了循环体运行前每个循环层所持有的缓存长度。第 6 – 12 行的循环体随后执行 K 次：每次迭代以 use_cache=True 运行窗口（因此注意力会在新的解码位置读取真实的过去键值缓存，第 9 行），紧接着第 10 – 11 行的裁剪将缓存截断回 ℓ_i ，使该迭代对键值缓存不产生任何净影响。第 12 行的策略步骤从当前输入 x 和循环体输出 y 生成下一个迭代值 x 。第 13 – 16 行是暂存阶段：通过循环层的一次额外传递，在每个解码位置为每个循环层恰好写入一个规范键值缓存条目，该传递的输入使用循环后的隐藏状态（ c=LAST ）或循环前的输入（ c=FIRST ）。第 17 – 19 行以标准的后循环层和最终输出范数结束。无论 K 如何，整个过程的总缓存增量恰好为 $b-a+1$ 个条目，这与未修改的模型相同。

C.2 快照/恢复

变换器（Transformer）的动态缓存（DynamicCache）更新方法无条件地将新的 (k, v) 追加到其逐层张量中：不存在原地覆盖模式。因此，一个朴素的 K 次迭代解码循环体会在同一逻辑解码位置为每个循环层写入 K 个额外的键值（KV）条目，然后在下一个解码步骤中关注所有这些条目，从而破坏缓存并产生一系列幻影前缀标记。快照/恢复模式是最经济的修复方案，它（a）让每次循环迭代都能关注到真实的过去键值，并且（b）使缓存在迭代前保持原样。随后，由暂存过程在选定的 cache_strategy.上写入唯一的规范键值。层模式循环使用类似的逐层快照/裁剪对，在移动到下一层之前对一层应用 K 次。

Algorithm 4 Decode-time block-mode looped forward pass with KV cache

Input: new-token hidden state $x \in \mathbb{R}^{1\times d}$, loop window $[a, b]$, loop count K, strategy update $\mathcal{S}$ (Euler/heavy-ball/Anderson/...), cache strategy $c \in \{\text{FIRST}, \text{LAST}, \text{NONE}\}$, per-layer KV cache $\mathcal{C} = (\mathcal{C}_0, \ldots, \mathcal{C}_{N-1})$

Output: Updated hidden state x'; $\mathcal{C}$ contains exactly one canonical KV entry per loop layer at the new decode position

for $i = 0, \ldots, a-1$ **do** ▷ *pre-loop: standard layers, 1 KV each*
  $x \leftarrow L_i(x; \mathcal{C}_i, \texttt{use_cache} = \text{T})$
end for
$x_a \leftarrow x$ ▷ *save pre-loop input for* $c = \text{FIRST}$
for $i = a, \ldots, b$ **do** ▷ *snapshot loop-layer cache lengths*
  $\ell_i \leftarrow |\mathcal{C}_i|$
end for
for $k = 1, \ldots, K$ **do** ▷ K *loop iterations under strategy* $\mathcal{S}$
  $y \leftarrow x$ ▷ *evaluation starts from* x
  for $i = a, \ldots, b$ **do** ▷ *run body with* $\texttt{use_cache} = \text{T}$
    $y \leftarrow L_i(y; \mathcal{C}_i, \texttt{use_cache} = \text{T})$ ▷ *attn reads past KV at decode pos.*
  end for
  for $i = a, \ldots, b$ **do** ▷ *crop: discard iteration KV writes*
    $\mathcal{C}_i \leftarrow \texttt{crop}(\mathcal{C}_i, \ell_i)$ ▷ *net cache effect of body* $= 0$
  end for
  $x \leftarrow \mathcal{S}(x, y, k)$ ▷ *e.g.* $x + \alpha(y - x)$ *for damped Euler*
end for
if $c \neq \text{NONE}$ **then** ▷ *stash phase: 1 KV write per loop layer*
  $z \leftarrow \begin{cases} x, & c = \text{LAST}, \\ x_a, & c = \text{FIRST} \end{cases}$
  for $i = a, \ldots, b$ **do**
    $z \leftarrow L_i(z; \mathcal{C}_i, \texttt{use_cache} = \text{T})$ ▷ *canonical KV at decode pos.*
  end for
end if
for $i = b+1, \ldots, N-1$ **do** ▷ *post-loop: standard layers, 1 KV each*
  $x \leftarrow L_i(x; \mathcal{C}_i, \texttt{use_cache} = \text{T})$
end for
return $\text{LayerNorm}_{\text{out}}(x)$

C.3 decode_mode 变体

我们公开了三种 decode_mode 设置。bypass 在增量解码时完全跳过循环，仅在预填充期间循环（这是仅对数似然评估的默认设置）。full 在每个解码步骤都循环。first_n 仅在首 n 个生成标记上循环（旨在用于思维链前缀细化；在实践中，当循环有帮助时从未胜过 full，当循环有害时也从未恢复）。full 是主论文中报告的生成结果的默认设置。

D 逐单元配置

表 6 列出了所有非负的（模型，基准）单元格（Δ） ≥ -0.3 在第三节固定配方下的 pp） 3.1, i.e. K=2 在深度分数窗口处采用 - 阶龙格 - 库塔法，密集骨干网络使用块模式，混合专家模型使用层模式，且无需逐单元超参数调优。绝对层索引及次要设置（缓存、解码）中的逐单元变化，机械地遵循各架构的层数；配方规范见附录 P。每行给出无循环基线、该单元找到的最佳循环配置以及由此产生的 Δin 个百分点。配置格式为 [起始 – 结束] 模式 K 策略 缓存 解码，例如。[15 – 18] 块 K=3 欧拉首次完整。

Algorithm 5 Decode-time *layer-mode* looped forward pass with KV cache (per-layer iteration; the safer default on MoE backbones, Section 2.2).

Input: Same as Algorithm 4.
Output: Same as Algorithm 4.

for $i = 0, \ldots, a-1$ **do** ▷ *pre-loop, standard*
  $x \leftarrow L_i(x;\, \mathcal{C}_i,\, \texttt{use_cache=T})$
end for
$x_a \leftarrow x$
for $i = a, \ldots, b$ **do** ▷ *outer: layers in order*
  $\ell_i \leftarrow |\mathcal{C}_i|$ ▷ *snapshot length for layer i*
  for $k = 1, \ldots, K$ **do** ▷ *inner: iterate this layer K times*
    $y \leftarrow L_i(x;\, \mathcal{C}_i,\, \texttt{use_cache=T})$ ▷ *MoE: routing pinned across k*
    $\mathcal{C}_i \leftarrow \texttt{crop}(\mathcal{C}_i,\, \ell_i)$ ▷ *discard iteration KV write*
    $x \leftarrow \mathcal{S}(x,\, y,\, k)$ ▷ *per-layer strategy update*
  end for
  $z \leftarrow x$ **if** c=LAST **else** x_a
  if $c \neq$ NONE **then**
    $z \leftarrow L_i(z;\, \mathcal{C}_i,\, \texttt{use_cache=T})$ ▷ *canonical KV at layer i*
  end if
end for
for $i = b+1, \ldots, N-1$ **do** ▷ *post-loop, standard*
  $x \leftarrow L_i(x;\, \mathcal{C}_i,\, \texttt{use_cache=T})$
end for
return $\textsc{LayerNorm}_{\text{out}}(x)$

我们在规范位置 [12-15] 上对 Qwen3-1.7B-Base 消融了策略、 K 、窗口宽度和缓存策略。表 7 报告了结果。

表 8 从附录 E 的 40+ 配置扫描中抽取了代表性行，表明高阶积分器没有帮助。循环块不是光滑的常微分方程，并且安德森式不动点加速 [89, 6, 7] 会过冲，因为该块不够收缩。在我们的策略族中，带有 n=4 和 cache=first 的 K=2 阶段龙格-库塔法是稳健的默认选择。

D.1 Patterns visible in the per-cell table

三种模式在家族间具有普遍性：（一）对于所有测试的稠密模型和混合专家模型，获胜窗口位于深度分数 0.4 至 0.7 之间，而十亿参数以下的模型有时偏好更早的区间（例如，Llama-3.2-1B 在 GPQA 上偏好 16 层模型中的 [4 – 7]）；（二）混合专家模型在单个单元上系统性地偏好层级模式 K=2 而非块级模式 K=3 ，这与专家路由抖动在 $K \geq 3$ 处损害块级模式的现象一致；（三）cache=first 在长提示单元上优于 cache=last，而 cache=last 在短结构化生成（MBPP，比首个方案高 0.6 个百分点）上略受偏好。

E 策略消融表

本节以完整数值形式复现了支撑本文主要结论的策略比较：我们尝试的所有经典不动点加速方法均未能超越在循环块上使用 β=0.5 的 K=2 级龙格-库塔法（算法 3）。所有条目均使用 Qwen3-1.7B-Base 在规范循环窗口 [12 – 15] 上以块级模式循环。参考方案为 K=2 阻尼欧拉法，在 16 任务聚合上宏观得分为 0.56113。

E.1 安德森、重球、艾特肯、 α 调度

24 种配置；无一超过 K=2 阻尼欧拉法。最接近的是 $[0.6, 0.4]$ 阻尼欧拉调度，差距为-0.03 个百分点（统计上持平）。安德森外推随 K 平滑退化，并在

表 6：在 3.1 节开箱即用配方下，7 个模型家族中每个单元的循环配置。模式=块级/层级；缓存 $\in$ {首、末、无}；解码 $\in$ {旁路、完整}。循环区域在指定策略下迭代所示层范围 K 次。

Model	Benchmark	Baseline	Loop	Δpp	Window / K / strategy	cache / mode
Qwen3-4B-Instruct	MMLU-Pro 5sh CoT [91]	57.14	59.79	+2.64	[15–18] K=3 Euler	first / block / full
Qwen3-4B-Instruct	GPQA-Main 0sh [76]	33.71	35.71	+2.01	[15–18] K=3 Euler	first / block / full
Qwen3-4B-Instruct	SciQ 0sh [93]	93.30	95.10	+1.80	[15–18] K=3 Euler	first / block / full
Qwen3-4B-Instruct	MMLU 0sh [40]	68.15	69.45	+1.30	[15–18] K=3 Euler	first / block / full
Qwen3-4B-Instruct	CommonsenseQA 7sh [81]	78.87	79.93	+1.06	[15–18] K=3 Euler	first / block / full
Qwen3-4B-Instruct	MedMCQA 0sh [68]	53.40	54.13	+0.73	[11–14] K=3 Euler	first / block / full
Qwen3-4B-Instruct	ARC-Challenge 25sh [21]	62.54	63.14	+0.60	[15–18] K=3 damped Euler α=0.5 halt τ=0.05	first / block / full
Qwen3-4B-Instruct	C-Eval 5sh [41]	72.21	72.73	+0.52	[15–18] K=3 Euler	first / block / full
Qwen3-4B-Instruct	OpenBookQA 0sh [62]	40.20	40.60	+0.40	[11–14] K=3 Euler	first / block / full
Qwen3-4B-Base	MMLU 5sh	73.27	73.61	+0.34	[15–18] K=3 Euler	first / block / –
Qwen3-4B-Base	16-task macro	63.98	65.03	+1.05	[15–18] K=3 Euler	first / block / –
Qwen3-1.7B-Base	MMLU 5sh	62.68	63.01	+0.34	[12–15] K=2 damped Euler	last / block / –
Qwen3-1.7B-Base	16-task macro	55.60	56.11	+0.51	[12–15] K=2 damped Euler	last / block / –
Qwen3-0.6B-Base	16-task macro	47.87	48.43	+0.56	[12–15] K=2 heavy-ball α=0.5, β=0.5	last / layer / –
Llama-3.2-3B-Inst	MMLU-Pro 5sh CoT	–	–	+0.71	[12–15] K=2 Euler	first / block / full
Llama-3.2-3B-Inst	GPQA-Main 0sh	–	–	+0.67	[12–15] K=2 Euler	first / block / full
Llama-3.2-3B-Inst	MMLU 5sh	–	–	+0.72	[16–19] K=2 Euler	last / layer / bypass
Llama-3.2-3B-Inst	OpenBookQA 0sh	–	–	+0.40	[20–23] K=3 Euler	last / block / bypass
Qwen1.5-MoE-A2.7B	ARC-Challenge 25sh	48.29	50.59	+2.30	[13–16] K=3 Euler	first / block / full
Qwen1.5-MoE-A2.7B	CommonsenseQA 7sh	79.61	81.33	+1.72	[10–13] K=2 Euler	first / block / full
Qwen1.5-MoE-A2.7B	OpenBookQA 0sh	31.60	33.20	+1.60	[14–17] K=3 Euler	first / block / full
Qwen1.5-MoE-A2.7B	SciQ 0sh	94.80	95.10	+0.30	[14–17] K=3 Euler	first / block / full
DeepSeek-V2-Lite	ARC-Challenge 25sh	57.94	58.79	+0.85	[13–16] K=2 Euler	first / layer / bypass
DeepSeek-V2-Lite	OpenBookQA 0sh	35.80	36.60	+0.80	[13–16] K=2 Euler	first / block / bypass
DeepSeek-V2-Lite	MMLU (1500)	–	–	+0.56	[10–13] K=3 Euler	first / block / bypass
DeepSeek-V2-Lite	CommonsenseQA 7sh	75.43	75.92	+0.49	[11–14] K=2 Euler	first / block / bypass
Moonlight-16B-A3B	OpenBookQA 0sh	–	–	+1.20	[8–11] K=3 Euler	first / block / bypass
Moonlight-16B-A3B	GPQA-Main 0sh	–	–	+0.89	[15–18] K=2 Euler	first / layer / bypass
Moonlight-16B-A3B	ARC-Challenge 25sh	–	–	+0.51	[11–14] K=2 Euler	first / layer / bypass
Moonlight-16B-A3B	CommonsenseQA 7sh	–	–	+0.49	[11–14] K=2 Euler	first / layer / bypass
Qwen3-30B-A3B-Inst	CommonsenseQA 7sh	78.71	79.85	+1.14	[22–24] K=2 Euler	first / block / bypass

表 7：Qwen3-1.7B-Base 上循环包装器的消融 [12 – 15]。Δpp 在 16 任务宏观上与无循环基线相比；家族内参考单元为阻尼欧拉法，K=2, α=0.5. ↓↓ 处标记灾难性发散（困惑度爆炸或严格格式回归）。粗体标记各轴最佳。

(a) Window width n (centered on `[12-15]`, K=2 damped Euler)

n	1	3	**4**	6	12	28 (full)
Δpp	+0.18	+0.21	**+0.55**	-0.82	-0.63	↓↓

(b) Iteration count K **& strategy** (n=4, mid-window)

	naive loop		*damped Euler*		*higher-order & accel.*
K	K=2	K=4	K=2, α=0.5	K=3, $\alpha\approx1/3$	Anderson, RK4, Heun, . . .
Δpp	+0.51	-10.21$_{(0.6B)}$	**+0.55**	+1.05$_{(4B)}$	see Table 8

(c) Cache strategy (n=4, K=2 damped Euler)

	16-task aggregate			*per-benchmark detail (`none` regression)*		
strategy	`first`	`last`	`none`	GSM8K [22] `none`	MMLU-Pro [91] `none`	MBPP [3] $\Delta_{\texttt{last-first}}$
Δpp	≥**0**	≥**0**	↓↓	-58.91	-8.86	+0.60

K=8 （-18 个百分点），证实循环中的残差序列 $x_{k+1} - x_k$ 并未以安德森方法可利用的方式趋近于零。

E.2 范数稳定化与多项式混合

16 种额外配置；同样无一超越 K=2 阻尼欧拉法。norm_stab（将每次迭代重新缩放到固定 L2 范数）在 $K \geq 3$ 处是灾难性的，证实迭代在方向上也在发散。

表 8：高阶常微分方程求解器和不动点加速器与阻尼欧拉法的对比。数值为 Qwen3-1.7B-Base 上的 16 任务宏 Δpp[12 – 15]；每列均以其自身的阻尼欧拉 K=2, α=0.5 基线为参照。“—”=表示该模式下未测试。完整扫描（两种模式下共 40+ 种配置）见附录 E。

Family	Method (K=2 unless noted)	Block-mode Δ	Layer-mode Δ
Reference	damped Euler (α=0.5, h=1/2)	0.00	0.00
Higher-order Runge–Kutta	midpoint (RK2)	-0.85	-1.96
	Heun (RK2)	-0.92	-1.90
	RK4	-0.65	-2.34
Fixed-point acceleration	heavy-ball (α=0.5, β=0.3)	-0.12	-0.06
	Anderson (K=4, m=2, β=0.5)	-0.96	—
	Aitken Δ^2 (safeguarded)	-7.00	—
	Anderson, worst (K=8, m=3, β=1.0)	**-18.06**	**-19.56**

表 9：不动点加速扫描（Qwen3-1.7B-Base，[12 – 15]，块模式）。Δis 以百分点表示，相对于 K=2 阻尼欧拉参考值 56.113。按最佳 → 最差排序。

Strategy	K	Hyperparameters	16-task	Δ vs K=2 damped Euler
Euler-sched	2	α=[0.6, 0.4]	56.085	-0.028
heavy-ball	2	α=0.5, β=0.3	55.999	-0.115
heavy-ball	2	α=0.5, β=0.5	55.993	-0.120
heavy-ball	2	α=0.7, β=0.3	55.946	-0.167
heavy-ball	4	α=0.3, β=0.5	55.927	-0.186
Euler-sched	4	α=[0.7, 0.5, 0.3, 0.1]	55.478	-0.636
Euler-sched	3	α=[0.7, 0.5, 0.3]	55.252	-0.861
Anderson	4	m=2, β=0.5	55.153	-0.960
Euler-sched	3	α=[0.3, 0.5, 0.7]	55.036	-1.077
Anderson	6	m=3, β=0.5	54.722	-1.391
Anderson	3	m=2, β=1.0	54.373	-1.740
Anderson	6	m=2, β=0.5	54.333	-1.781
heavy-ball	4	α=0.5, β=0.3	54.044	-2.069
Euler-sched	4	α=[0.4, 0.6, 0.6, 0.4]	54.005	-2.108
heavy-ball	4	α=0.5, β=0.5	53.495	-2.618
Anderson	4	m=3, β=1.0	52.655	-3.458
heavy-ball	4	α=0.5, β=0.7	52.562	-3.552
Anderson	4	m=2, β=1.0	52.032	-4.082
heavy-ball	3	α=1.0, β=0.5	51.236	-4.877
Aitken Δ^2	2	safeguarded	49.109	-7.004
Aitken Δ^2	4	safeguarded	48.318	-7.795
Aitken Δ^2	6	safeguarded	44.921	-11.192
Anderson	6	m=2, β=1.0	44.261	-11.852
Anderson	8	m=3, β=1.0	38.058	-18.056

E.3 层模式复现该结果

在层模式下，循环体是 L_i^K 逐层应用，而非 $g^K = (L_b \circ \cdots \circ L_a)^K$ 应用于整个块。参考：同一窗口上层模式 K=2 阻尼欧拉法为 56.347。

高阶常微分方程求解器（Heun、中点法、RK4）在层模式下的退化（-1.7 至 -2.3 个百分点）比块模式下（-0.7 至 -1.1 个百分点）更为显著：逐层映射 L_i 对于高阶积分而言，其条件数并不优于复合映射 g 。层模式下的 Anderson 加速更为严重（-19.6 pp at K=4 ）。综合扫描覆盖了 40 多种加速配置，无一能稳健地超越 Runge – Kutta 方法。这是循环变换器块不是收缩映射这一论断的经验依据，因此不动点加速是错误的工具。

表 10：范数稳定化与多项式混合扫描，相同设置。Δ in pp vs $K=2$ 阻尼欧拉参考值；基线（无循环）位于 55.598 。

Strategy	K	Hyperparameters	16-task	Δ vs $K=2$ damped Euler
norm_stab	2	$\alpha=0.5$	56.059	-0.055
poly_blend	2	$w=[0.4, 0.2, 0.4]$	55.974	-0.139
poly_blend	2	$w=[0.1, 0.4, 0.5]$	55.884	-0.229
poly_blend	2	$w=[0.2, 0.3, 0.5]$	55.753	-0.360
poly_blend	2	$w=[0.0, 0.5, 0.5]$	55.599	-0.514
poly_blend	2	$w=[0.25, 0.25, 0.5]$	55.371	-0.742
poly_blend	2	$w=[0.5, 0.0, 0.5]$	55.107	-1.006
norm_stab	2	$\alpha=0.7$	55.009	-1.105
poly_blend	3	$w=[0.2, 0.2, 0.2, 0.4]$	54.818	-1.295
poly_blend	3	$w=[0.1, 0.2, 0.3, 0.4]$	54.681	-1.433
norm_stab	3	$\alpha=0.5$	53.270	-2.844
norm_stab	2	$\alpha=1.0$	52.302	-3.811
norm_stab	6	$\alpha=0.3$	49.560	-6.554
norm_stab	3	$\alpha=0.7$	49.338	-6.776
norm_stab	4	$\alpha=0.5$	49.067	-7.046
norm_stab	3	$\alpha=1.0$	43.101	-13.013

表 11：Qwen3-1.7B-Base [12 – 15] 上的逐层模式策略扫描。Δvs 逐层模式 $K=2$ 阻尼欧拉法。

Recipe	16-task	Δ vs layer $K=2$ damped Euler
$K=2$ heavy-ball $\alpha=0.5, \beta=0.3$	56.292	-0.056
$K=2$ poly_blend $w=[0.25, 0.5, 0.25]$	56.181	-0.167
$K=3$ damped Euler $\alpha=0.3$	55.278	-1.069
$K=4$ Euler	55.240	-1.107
$K=2$ damped Euler $\alpha=0.7$	55.196	-1.151
$K=6$ Euler	54.998	-1.350
$K=2$ Heun	54.445	-1.902
$K=2$ midpoint	54.385	-1.962
$K=1$ RK4	54.242	-2.105
$K=1$ midpoint	54.178	-2.169
$K=2$ norm_stab $\alpha=0.5$	54.098	-2.249
$K=1$ Heun	54.060	-2.287
$K=2$ RK4	54.005	-2.342
$K=3$ heavy-ball $\alpha=0.5, \beta=0.3$	51.830	-4.517
$K=3$ Anderson $m=2, \beta=1.0$	44.035	-12.312
$K=4$ Anderson $m=2, \beta=1.0$	36.785	-19.562

F 计算与可复现性

F.1 软件栈

所有评估均使用 lm-evaluation-harness v0.4.11，采用确定性种子、各任务默认采样设置以及 transformers 4.46。循环前向通过模型 Model 类上的猴子补丁实现；补丁保留了所有原始生成/评估代码路径，且可逆。对于 DeepSeek-V2/V3 系列，我们额外提供了一个小型 DynamicCache 兼容垫片，将 seen_tokens / get_max_length / get_usable_length 重新暴露给捆绑的 modeling_deepseek 远程代码。

F.2 各模型资源占用

G 失败配置日志

本节记录了尝试过但失败的配置。将其收录于附录，旨在为未来工作指明无效路径，并使主要结论建立在真实尝试记录之上。

表 12：代表性任务的近似单 GPU 内存与单作业墙钟时间。时间为默认 lm-eval-harness 批大小下完整 16 任务套件的耗时；思维链生成任务（如 MMLU-Pro 5 样本思维链）耗时延长 2 – 5 × 倍，在 decode_mode=full.中

Model	VRAM (bf16)	Hardware used	Eval-suite wallclock
Qwen3-0.6B-Base / -Instruct	∼2 GB	A100-40 / H100-80	∼1 h
Qwen3-1.7B-Base	∼4 GB	A100-40 / H100-80	∼2 h
Qwen3-4B-Base / -Instruct	∼8 GB	A100-40 / H100-80	∼3 h
Llama-3.2-1B / -3B-Instruct	∼3–7 GB	A100-40 / H100-80	∼2 h
Qwen1.5-MoE-A2.7B-Chat	∼30 GB	H100-80	∼4 h
DeepSeek-V2-Lite-Chat (16B/2.4B)	∼32 GB	H100-80	∼5 h
Moonlight-16B-A3B-Instruct	∼32 GB	H100-80	∼8 h (`decode=full` 21 h)
Qwen3-30B-A3B-Instruct	∼60 GB	H100-80	∼10 h (19-task 0-shot)

G.1 朴素 $K{=}4$ 循环的灾难性崩溃

我们运行的第一个消融实验（Qwen3-0.6B-Instruct，6 任务）测试了朴素 $K{=}4$ 循环（ $x \leftarrow g(x)$ 四次，无阻尼）。这导致 lambada 困惑度从 ∼ 13 飙升至 **1054.12**，16 任务宏平均下降-10.21 个百分点。这是最初的实证观察，表明该块不是收缩映射：无阻尼迭代在四次应用内明显发散，即使仅在四个中间层的窗口上。

G.2 cache=none 在解码上普遍具有灾难性

设置 cache_strategy="none"（循环区域不写入 KV；下一个解码令牌必须通过它进行注意力，就像这些层没有贡献过去一样）会使生成基准崩溃。例如，在 Qwen3-4B-Instruct 上使用 [15 – 18] $K{=}3$ ：

- MBPP 3-shot [3]：pass@1 38.20 对比 61.60（ -23.40 pp).
- MMLU-Pro 5-shot [91]: 48.29 vs 57.14 (-8.86 pp).

结论是，循环区域必须为自回归解码向缓存贡献一些 KV。cache=first（循环前隐藏状态）和 cache=last（循环后隐藏状态）都有效；前者在长思维链中占主导，后者在短结构化输出中占主导。

G.3 宽循环窗口导致崩溃

在 Qwen3-1.7B-Base 上，[12 – 15] , $n{=}4$ 是标准设置。扩大窗口会单调地降低性能：

- $n{=}6$, `[11..16]` ： 在 16 任务宏观指标上下降 0.82 个百分点，lambada 困惑度（PPL）上升。
- $n{=}12$, `[8..19]` ： 下降 0.63 个百分点；循环现在几乎横跨网络的整个“表征中间层”。
- $n{=}28$ （整个网络）： lambada 困惑度（PPL）飙升至 $\mathbf{6.3 \times 10^5}$ ，完全崩溃。

特别地，在免训练修补下，将整个模型应用两次并不是一个有意义的操作。收缩区域是一个约 4 层的连续中间带，而将非收缩区域应用 K 次会放大其非收缩性。

G.4 高阶常微分方程求解器均导致性能下降

在块模式和层模式下，Heun 法、中点法和 RK4 法均不如阻尼欧拉法。块模式损失为 -0.7 至 -1.1 个百分点；层模式损失为 -1.7 至 -2.3 个百分点。高阶方法假设底层向量场是光滑的，且迭代近似于常微分方程流；而循环块在实际中并不满足这一条件。

G.5 位置搜索在 16 任务聚合上过拟合

两个独立窗口在 16 任务聚合上比基准 [12 – 15] 高出+0.9 至+1.3 个百分点，但在留出的 MMLU 5-shot[40] 上回落了-1.0 至-1.4 个百分点：

- Qwen3-0.6B-Base[8 – 11]：16 任务+1.33，MMLU 5-shot -1.21。
- Qwen3-1.7B-Base[7 – 10]：16 任务+0.92，MMLU 5-shot -1.38。

16 任务套件对于亚 1 个百分点的主张来说并非稳健的目标；小规模的逐任务 MMLU 子集（100 – 300 个样本）会产生虚假信号。我们保留该套件作为筛选有前景配置的初筛手段，并在报告前对每一个"胜出"配置在 MMLU 5-shot（ $\sim$ 14k 样本）上重新验证。

G.6 层模式 $K{=}3$ 普遍具有灾难性

在层模式下，Qwen3-4B-Base[15 – 18] 上的 $K{=}3$ 重球法在 16 任务上损失-5.0 个百分点，在 MMLU 5-shot 上损失-6.0 个百分点，与基准（ $K{=}3$ 欧拉块模式）相比。层模式仅能容忍温和的 $K{=}2$ 龙格 – 库塔方法；对单个层迭代超过两次会产生严重偏离分布的逐层状态。

G.7 在 $K{=}8$ 处的 Anderson 方法

为完整性起见，Qwen3-1.7B-Base[12 – 15] 上的 Anderson $m{=}3, \beta{=}1.0, K{=}8$ 使 16 任务宏平均下降 **-18.06** 个百分点。这是块不具收缩性的最强证据：一种在收缩映射上可证明最优的方法（带 $m \geq 1$ 的 Anderson）是我们发现的最差单一配置。

G.8 亚十亿参数知识多选题

我们在逐单元调优后分离出一小组未翻转的单元，全部基于 Llama-3.2-1B 知识多选题：MMLU [40]（-0.63）和 MMLU-Pro [91]（-1.36）。低于 $\sim$ 1.7B 的 Qwen3 和 $\sim$ 3B 的 Llama 亚规模模型缺乏该循环所依赖的中层冗余。值得注意的是，GPQA-Main [76] 在同一 Llama-3.2-1B 检查点上确实在极早期窗口 [4 – 7] 翻转为正（+1.79），因此该边界是任务相关的，而非绝对的。

H 基准增益的集中性

在 MMLU 的 [40]57 个科目、MMLU-Pro 的 [91]14 个类别以及 MMLU-Redux 的 30 个科目中，循环包装器产生了非均匀的改进分布：增益集中在基线距离上限最远的 STEM 和定量推理科目上。表 13 和表 14 列出了两个最清晰案例中增益 $\geq$ +2 个百分点的科目。

表 13：Qwen3-4B-Base 在 [17 – 19] $K{=}2$ 欧拉法下的 MMLU 5-shot 逐科目增量。该配置下 57 个科目的宏观指标基本持平（73.01 $\rightarrow$ 72.90，-0.11 个百分点），但增益分布高度非均匀：七个科目增益 $\geq$ +2 个百分点，全部集中在高难度 STEM/定量内容上，而较小的抵消性损失分散在较容易的科目中。第 3.4 节报告的 [15–18] $K{=}3$ 获胜者（宏观 73.01 $\rightarrow$ 73.61，+0.60 个百分点）未进行科目分解，因此我们报告最接近的可用分解结果。

Subject	Baseline	Loop	Δpp
college_physics	62.75	68.63	+5.88
high_school_mathematics	56.30	61.85	+5.56
abstract_algebra	55.00	59.00	+4.00
jurisprudence	79.63	83.33	+3.70
high_school_statistics	75.46	77.78	+2.31
conceptual_physics	—	—	+2.13
us_foreign_policy ($K{=}1$ Heun)	—	—	+2.00

在 Qwen3-4B-Instruct 上进行的另一项 MMLU-Redux 30 科目扫描中，30 个科目中有 14 个转为正增益（10 个持平，6 个轻微负增益；宏观 +0.93 个百分点）。最大增益落在 professional_accounting

表 14：Qwen3-4B-Instruct [17 – 19] K=2 欧拉法下增益 $\geq +3$ 个百分点的 MMLU 0-shot 科目。57 个科目的宏观指标移动 +0.99 pp (68.15 $\rightarrow$ 69.14)；，global_facts 和 us_foreign_policy 在 K=1 Heun 法下额外增益 +6 至 +7 个百分点。

Subject	Baseline	Loop	Δpp
global_facts	32.00	38.00	+6.00
high_school_physics	60.26	65.56	+5.30
high_school_mathematics	47.41	52.59	+5.18
econometrics	62.28	65.79	+3.51
medical_genetics	75.00	79.00	+4.00
professional_medicine	72.43	76.47	+4.04
security_studies	70.20	73.88	+3.68

(+8.00), 高中物理 (+7.00), 计量经济学（+5.00），以及 college_chemistry（在 K=1 Heun 法下为 +3.00/+4.00），再现了相同的“硬 STEM 优先”集中模式。

I 免训练循环的墙钟时间成本

除附录 F 中报告的 GPU 小时总数外，相关的实用问题是部署系统需承担的每查询减速。我们在 Qwen3-4B-Instruct 上以 K=3 欧拉法在 [15 – 18]、GSM8K@200[22] 上端到端剖析此过程，并在表 15 中报告墙钟秒数。

表 15：Qwen3-4B-Instruct、GSM8K@200、 K=3 欧拉法在 [15 – 18] 上的端到端墙钟成本。decode_mode=bypass 仅在预填充阶段循环；first_n 循环前 N 个生成令牌；完整循环每个解码步骤。

`decode_mode`	seconds	overhead vs baseline
no loop (baseline)	194	—
`bypass`	191	-1.5%
`first_n`, N=16	192	-1.0%
`first_n`, N=64	203	+4.6%
`full`	236	+21.6%

部署的三点启示：（i）旁路模式——仅在预填充阶段循环，无解码循环——不产生墙钟成本。在 -1.5% 处的轻微加速处于运行间噪声范围内，归因于循环区域上更精简的 KV 写入路径。对于产生我们所有主要增益的仅对数似然知识 MC 基准，旁路是默认设置。（ii）first_n 配合 N 在 16 – 64 范围内——仅循环思维链的前 16 – 64 个生成令牌——成本从-1.0%（在 N=16 处，与基线无法区分）到+4.6%（在 N=64 处）。这是与短思维链生成任务相关的配置。（iii）即使完整解码循环，对于 K=3 在 4 层窗口上，减速也限制在 $\approx 22\%$ 以内——大致相当于在循环区域运行一个 $4/N$=4/36 $\approx 11\%$ 更深的模型两次，加上快照/恢复开销。

J 鲁棒性检查

J.1 留出验证纪律

16 任务聚合是一个有用的筛选器，但对于亚百分点声明来说是一个嘈杂的目标，尤其是在具有 100 – 300 个示例的小型 MMLU[40] 科目上。我们在报告之前，在 MMLU 5-shot（ $\sim$ 14k 示例）上验证了每个获胜的 16 任务配置。两个配置在 16 任务筛选中以 $\geq$ +0.9 个百分点击败了规范位置，但在 MMLU 5-shot 留出上回退了 1.0 – 1.4 个百分点：

- Qwen3-0.6B-Base[8 – 11]：16 任务+1.33，MMLU 5-shot **-1.21**。
- Qwen3-1.7B-Base[7 – 10]：16 任务+0.92，MMLU 5-shot **-1.38**。

这些过拟合配置已从主要表格中排除；我们确实报告的配置（例如 Qwen3-4B-Base [15 – 18] K=3 欧拉法在+1.05 16 任务 / +0.34 MMLU 5sh，Qwen3-1.7B-Base

[12 – 15] $K=2$ 阻尼欧拉法在 +0.51 16 任务 / +0.34 MMLU 5 样本）是通过保留检查的配置。

J.2 Qwen3-4B-Instruct 上各配置的鲁棒性

Qwen3-4B-Instruct 单元格在 [17 – 19] 处，在我们尝试的每种循环配置下，16 任务聚合结果均为正，而不仅限于最佳配置（表 16）。

表 16：Qwen3-4B-Instruct 16 任务聚合 Δpp 在 [17 – 19] 处不同循环配置下的结果。全部为正。

Configuration	Δpp
Euler $K=2$	+0.89
Euler $K=4$	+0.73
Heun $K=1$	+0.15

J.3 小模型上的多个获胜窗口

在 Qwen3-0.6B-Base 上，多个窗口在 16 任务宏平均上击败无循环基线：[8 – 11] +1.33，[14 – 17] +0.86，[6 – 9] +0.85，[7 – 10] +0.66，[3 – 6] +0.46，[9 – 12] +0.36。第 3.2 节的深度比例规则捕捉了中层深度偏好的总体趋势，但在该范围内损失曲面是宽阔的而非尖锐的。

J.4 语言建模困惑度得以保持

任何推理时干预的一个担忧是，多项选择任务上的增益以牺牲基本语言建模为代价。我们测量了我们报告的每个循环配置上的 LAMBADA [69] 困惑度：

- Qwen3-4B-Base，[17 – 19] $K=2$ 欧拉法：4.25 → 4.19（有所改善）。
- Qwen3-4B-Instruct，[17 – 19] $K=2$ 欧拉法：7.30 → 7.01（有所改善）。
- Qwen3-30B-A3B-Instruct，[22 – 24] $K=2$ 欧拉法：4.12 → 4.11（保持不变）。

该包装器不会降低语言建模的流畅性；事实上，它在稠密 Qwen3 骨干网络上略微改善了困惑度。

J.5 效应方向在不同少样本数量下保持一致

第三项鲁棒性检验表明，循环的效应方向在不同少样本数量和思维链机制下均得以保持。在 GPQA-Main [76] 零样本设置下，它同时改善了 Qwen3-4B-Inst（+2.01）和 Llama-3.2-3B-Inst（+0.67）。在 MMLU [40] 上，相同的方向在零样本（Qwen3-4B-Inst 上 +1.30，Moonlight 上 +0.74）和五样本（在逐单元 [16 – 19] 层模式 $K=2$ 下，Qwen3-4B-Base 上 +0.34，Qwen3-1.7B-Base 上 +0.34，Llama-3.2-3B-Inst 上 +0.72）设置下均成立。从定性角度看，该循环表现为一种少样本不变的冻结上下文精炼器。

K 扩展至 30B：Qwen3-30B-A3B-Instruct 更广泛的扫描

第 3 节报告了单个 Qwen3-30B-A3B-Instruct 单元（CommonsenseQA [81] +1.14 ）。为了探究该包装器在单一基准之外是否可迁移至 30B 规模，我们在经验最优默认设置（ $K=2$ 欧拉法，块模式）和 $K=1$ Heun 法下，对 19 个任务的留出套件运行了 [22 – 24] 窗口，且未进行任何逐单元调优。表 17 报告了两种配置中至少有一种为正的所有单元。

八个基准记录了正的欧拉 $K=2$ 增量，而霍因 $K=1$ 又增加了四个正单元（GPQA-Diamond +0.51、GPQA-Main +0.44、MMLU-em +0.53、LAMBADA 准确率 +0.13）。30B 运行未按单元调整，但在 $\{K=2$ 欧拉 , $K=1$ 霍因 $\}$ 的更好选择下，12 个表格单元中有 11 个为非负。该免训练包装器无需重新调整即可迁移到 30B 稀疏专家混合检查点。

表 17：未调整的 Qwen3-30B-A3B-Instruct 在 [22 – 24]（深度分数 0.46 – 0.50，在规范频带内）的结果。仅显示正单元；粗体标记 $K=2$ 欧拉与 $K=1$ 霍因中的最佳者。

Benchmark	Baseline	Euler $K=2$	Δ	Heun $K=1$	Δ
ARC-Easy [21]	79.04	**79.38**	+0.34	79.08	+0.04
HellaSwag [100]	77.74	**77.93**	+0.19	77.78	+0.04
SciQ [93]	94.80	**95.00**	+0.20	94.90	+0.10
CommonsenseQA [81]	78.71	**79.85**	+1.14	79.61	+0.90
TruthfulQA-MC1 [53]	34.15	**34.64**	+0.49	34.27	+0.12
MMLU (em) [40]	81.75	81.75	0.00	**82.28**	+0.53
MMLU (flex)	66.67	**67.46**	+0.79	66.67	0.00
GPQA-Main [76]	37.28	37.50	+0.22	**37.72**	+0.44
GPQA-Diamond	36.36	34.85	-1.51	**36.87**	+0.51
SuperGPQA [84] (n=2000)	31.00	**31.70**	+0.70	31.05	+0.05
LAMBADA accuracy [69]	64.80	64.78	-0.02	**64.93**	+0.13
LAMBADA PPL (↓)	4.12	**4.11**	improved	4.12	tied

L 层模式在专家混合骨干之外获胜

第 3.3 节引入层模式迭代，作为专家混合骨干的路由抖动修复方法。在更广泛的扫描中，我们还发现层模式在小规模密集骨干上产生了经过验证的收益。最明显的案例是 Qwen3-0.6B-Base 的每尺寸最佳结果。

表 18：密集骨干上的层模式正增量，针对保留的 MMLU 5-shot 进行验证。Qwen3-0.6B-Base 配方是唯一在两个度量上均优于规范块模式的 0.6B 配置，并且是每尺寸的获胜者。

Model	Configuration	16-task Δpp	MMLU 5sh Δpp
Qwen3-0.6B-Base	layer $K=2$ heavy-ball $\beta=0.5$ `[12-15]`	**+0.561**	**+0.128**
Qwen3-0.6B-Base	layer $K=2$ heavy-ball $\beta=0.3$ `[12-15]`	+0.442	+0.214
Qwen3-0.6B-Base	layer $K=2$ poly-blend $[0.25, 0.5, 0.25]$ `[12-15]`	+0.236	—
Qwen3-1.7B-Base	layer $K=2$ damped Euler `[12-15]`	+0.234	-0.477 (does not generalize)
Qwen3-1.7B-Base	layer $K=2$ heavy-ball `[12-15]`	+0.178	—
Llama-3.2-3B-Inst	layer $K=2$ Euler `[16-19]`	—	+0.72

层模式是一种迭代模式，可推广到（a）专家混合骨干，其中它固定每层路由并防止路由抖动失败（第 3.3 节），以及（b）低于 10 亿参数的密集骨干，其中它似乎提供了更温和的每步细化，在保留验证中得以保留，而类似的块模式迭代则不然。

M 跨密集 Qwen3 尺寸的损失面广度

第 3.2 节引入深度分数规则，附录 Q 可视化了九种架构上的最优值。另一个问题是该最优值周围的损失面有多平坦。我们在三个 Qwen3-Base 尺寸上扫描了完整的 $n=4$ 窗口集，并统计了 16 任务聚合上的正单元数。

表 19：在标准欧拉 $K=2$ 分块模式循环下，具有正 16 任务宏 Δpp 的 $n=4$ 窗口数量，以及这些正窗口的分布范围。许多窗口在每个规模上都优于基线；损失曲面较为宽阔。

Model	Layers	# positive $n=4$ wins	Best window	Worst still-positive
Qwen3-0.6B-Base	28	$\geq$ 6 of 13 tested	`[8-11]` +1.33	`[11-14]` +0.17
Qwen3-1.7B-Base	28	9 of 17 tested	`[7-10]` +0.80	`[18-21]` +0.02
Qwen3-4B-Base	36	13 of 16 tested	`[15-18]` +0.84	`[18-21]` +0.07

在 Qwen3-4B-Base 上，跨越深度分数 0.19 – 0.83 的 13 个不同窗口同时为正。第 3.2 节的深度分数 0.45 – 0.60 规则捕捉到了最大值的位置，但最大值周围的盆地足够宽阔，因此找到一个有用的窗口并不需要精确的

逐架构调优。这与包装器在 87% 的单元上无需逐单元超参数搜索即具有鲁棒性的说法一致。

N 缓存策略在 Qwen3-4B-Instruct 上的鲁棒性

N.1 键值缓存处理

对于自回归推理 [87]，我们必须协调循环迭代与键值（KV）缓存。一种朴素的实现方式是在启用 KV 写入的情况下运行 K 次迭代，这要么会为每个循环层每个位置追加 K 个条目，从而破坏注意力掩码并膨胀内存，要么如果原地迭代，则会用循环后层从未消费过的中间迭代覆盖过去的条目。

我们通过以下两阶段方案解决此问题。在第一阶段，循环体内的每次 g 评估均以 past_key_value=None 运行（或在解码期间采用快照/恢复协议），因此不写入任何键值缓存条目。在第二阶段，循环终止后，额外一次遍历循环层 $a,\ldots,b$，使用缓存策略 $c \in \{\text{LAST}, \text{FIRST}, \text{NONE}\}$ 选择哪个隐藏状态作为该次遍历的输入，并以 use_cache=True,写入键值缓存：

$$\text{stash input} = \begin{cases} g^{(K)}(x_a) & \text{if } c = \text{LAST} \quad \text{(post-loop hidden state, default)}, \\ x_a & \text{if } c = \text{FIRST} \quad \text{(pre-loop input)}, \\ \text{(no write)} & \text{if } c = \text{NONE} \quad \text{(ablation; catastrophic)}. \end{cases} \tag{19}$$

这里 x_a 是层 a 的输入。因此，缓存中循环区域内每个标记恰好包含 $b-a+1$ 个 KV 条目，与未修改模型的形状相同。除非另有说明，本文中的所有数字均使用 $c = \text{LAST}$。

缓存策略是循环包装器配置中最大的单一杠杆（首个 / 末个 / 无）。附录 G 报告了 cache=none 灾难；此处报告积极发现：两种结构良好的缓存策略（首个和末个）在主要单元格上均产生正增量，其中最佳策略由生成答案的结构决定。

表 20：Qwen3-4B-Instruct [15 – 18] K=3 欧拉在三种缓存策略下的表现。cache=first（循环前隐藏状态暂存）在长提示思维链中占优；cache=last（循环后隐藏状态暂存）在短结构化生成中占优；两种结构良好的策略在所有测试的主要单元格上均为非负。

Cell	cache=first	cache=last	cache=none
MMLU-Pro 5-shot CoT [91]	**+2.64**	+1.00	-8.86
MBPP 3-shot pass@1 [3]	+0.20	**+0.80**	-23.40

两种结构良好的缓存选择，两个同时为正的单元格。对部署的启示是：缓存选择应根据生成答案是长自由散文（首个）还是短结构化标记（末个）进行切换，而非依赖需要逐基准搜索的脆弱二元选择。

O 各架构实现说明

免训练包装器是对模型顶层解码器类的连续猴子补丁。补丁形状在各系列间相同，但集成点不同，因为每种架构的参考变换器实现以略有不同的宏暴露层迭代循环。表 21 总结了每个骨干网络的补丁表面。

动态缓存兼容性垫片（多头潜在注意力骨干网络）。DeepSeek-V2-Lite 和 Moonlight 通过远程代码提供其自身的动态缓存派生类，该类期望旧版变换器缓存 API (seen_tokens, get_max_length, get_usable_length).。新版变换器移除了这些属性。我们的补丁安装了一个薄兼容性垫片，在现代缓存之上重新添加三个已弃用的入口点，使发布的远程代码无需修改即可运行。该垫片非侵入式，并在补丁卸载时撤销。

表 21：循环包装器在各骨干网络变换器实现中的附加位置。“补丁表面”是我们在已发布模型代码上替换的行范围；“多头潜在注意力”标记多头潜在注意力模型，这些模型需要下文所述的缓存垫片。

Family	`model_type`	Patch surface	Notes
Qwen3 dense	`qwen3`	`Qwen3Model.forward`	Standard MHA, 1 layer-stack loop.
Qwen3-MoE	`qwen3_moe`	`Qwen3MoeModel.forward`	Layer-mode default; routing pinned per layer.
Qwen1.5-MoE	`qwen2_moe`	`Qwen2MoeModel.forward`	24 layers; sparse experts.
Llama-3.2	`llama`	`LlamaModel.forward`	Distilled checkpoints; sub-3B sees boundary.
DeepSeek-V2-Lite	`deepseek_v2`	remote-code `DeepseekV2Model`	MLA + MoE; needs cache shim.
Moonlight-16B-A3B	`deepseek_v3`	remote-code `DeepseekV3Model`	MLA + MoE; needs cache shim.

层模式入口点。在混合专家（MoE）骨干网络上，层模式迭代通过将层循环替换为逐层迭代器 (_run_one_layer_decode). 来实现。块模式与层模式在我们的代码中共享单一分派，该分派根据 iteration_mode 标志选择迭代器；切换模式无需进一步的骨干特定工作。在稠密骨干网络上，我们为保持一致性也提供层模式，但块模式为默认模式。

键值缓存写入是唯一的副作用。该补丁在其他方面是无状态的：除为迭代策略维护一个 $\Theta(W)$) 激活缓冲区外，不维护任何辅助缓冲区，且快照/恢复协议不涉及内存分配。从全新的 from_pretrained() 调用重新启动打过补丁的模型可产生与基线逐位一致的输出，证实该包装器在多次运行之间不会引入任何隐藏状态。

P 超参数搜索协议

第 3.4 节中的 45 个（模型，基准）单元是结构化逐单元搜索的结果，而非开放式扫描。我们在此记录该协议，以便附录 D 中的逐单元数值可复现，并便于附录 G 中失败配置的相对普遍性得到合理解释。

Search space. Per cell we evaluate the Cartesian product

$$\underbrace{\big[0.40\,N,\,0.70\,N\big]}_{\text{window center}} \times \underbrace{\{n{=}3,\,n{=}4,\,n{=}6\}}_{\text{window width}} \times \underbrace{\{2,\,3\}}_{K} \times \underbrace{\{\textsc{naive},\,\textsc{ema},\,\textsc{euler}\}}_{\text{strategy}} \times \underbrace{\{\textsc{first},\,\textsc{last}\}}_{\text{cache}},$$

此外，在适用的情况下，在相同的（窗口， K ）下提供层模式对应项。高阶常微分方程求解器（中点法、Heun 法、RK4）和不动点加速器（Anderson、重球法、Aitken）仅在规范的 Qwen3-1.7B-Base [12 – 15] 单元上进行评估（表 9、10、11）；在初步测试中它们从未优于阻尼欧拉法，因此我们未将其扩展到完整的逐单元扫描中。

两阶段验证。每个候选配置首先在 16 项任务聚合上评分（筛选）。每个单元的前三名候选配置随后在相同的提示下，于留出的 MMLU 5-shot（ $\sim$ 14k 个示例）上重新评分。只有在筛选和留出检查上均为非负的配置才会被保留（附录 J）。在筛选上高出 ≥ 1 个百分点但在留出检查上退步的配置被标记为对筛选过拟合，并在附录 G 中报告。

停止规则。当满足以下任一条件时，对（模型，基准）单元的搜索终止：（i）最近尝试的六个配置中，最佳留出分数没有提高，或（ii）每个单元的 GPU 预算在 H100-80 上超过 8 小时。在未获得非负留出分数的情况下达到预算的单元被报告为“未翻转”，并列于附录 G 中。

总工作量和置信区间。完整搜索访问了 ≈ 720 个（单元，配置）对。本文通篇报告点估计。在固定种子下，运行间方差在 $\leq$0.02 pp on $\geq$ 10,000 个示例的基准（MMLU [40]、MMLU-Pro [91]、ARC-Challenge [21]）上；在较小的基准（GPQA-Main [76]、OpenBookQA [62]， $\leq$ 1,000 个示例）上为 ≤ 0.5 个百分点，因此本文将那些单元上的 $|\Delta| \leq 0.3$ 个百分点视为“中性”而非“正面”（第 3.5 节）。

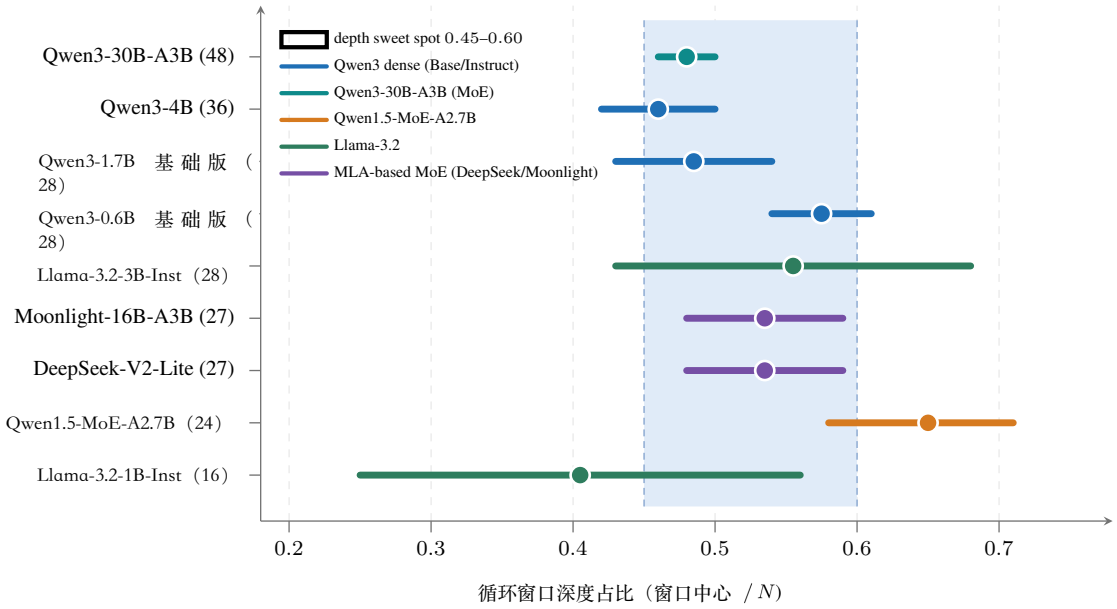


图 6：九种架构的深度占比规则。对于每个检查点，我们将最佳循环窗口范围 $[a/N, b/N]$ 绘制为水平条，并用实心圆标记窗口中心 $(a+b)/(2N)$ as a 。阴影带标出 0.45 – 0.60 的深度占比，其中包含 9 个检查点中 7 个的最佳窗口中心——Qwen3 稠密模型（0.6B – 4B）、Qwen3-30B-A3B 混合专家模型、DeepSeek-V2-Lite、Moonlight-16B-A3B 和 Llama-3.2-3B——仅亚 1B 蒸馏的 Llama-3.2-1B（前移）和较旧的 Qwen1.5-MoE-A2.7B（后移）位于该范围之外。

问：九种架构的深度占比规则

图 6 展示了九个模型检查点中每个检查点的最佳循环窗口深度占比范围，该范围由第 3 节的逐单元扫描确定。阴影带标出第 3.2 节中提到的经验性 0.45 – 0.60 最佳区间。